

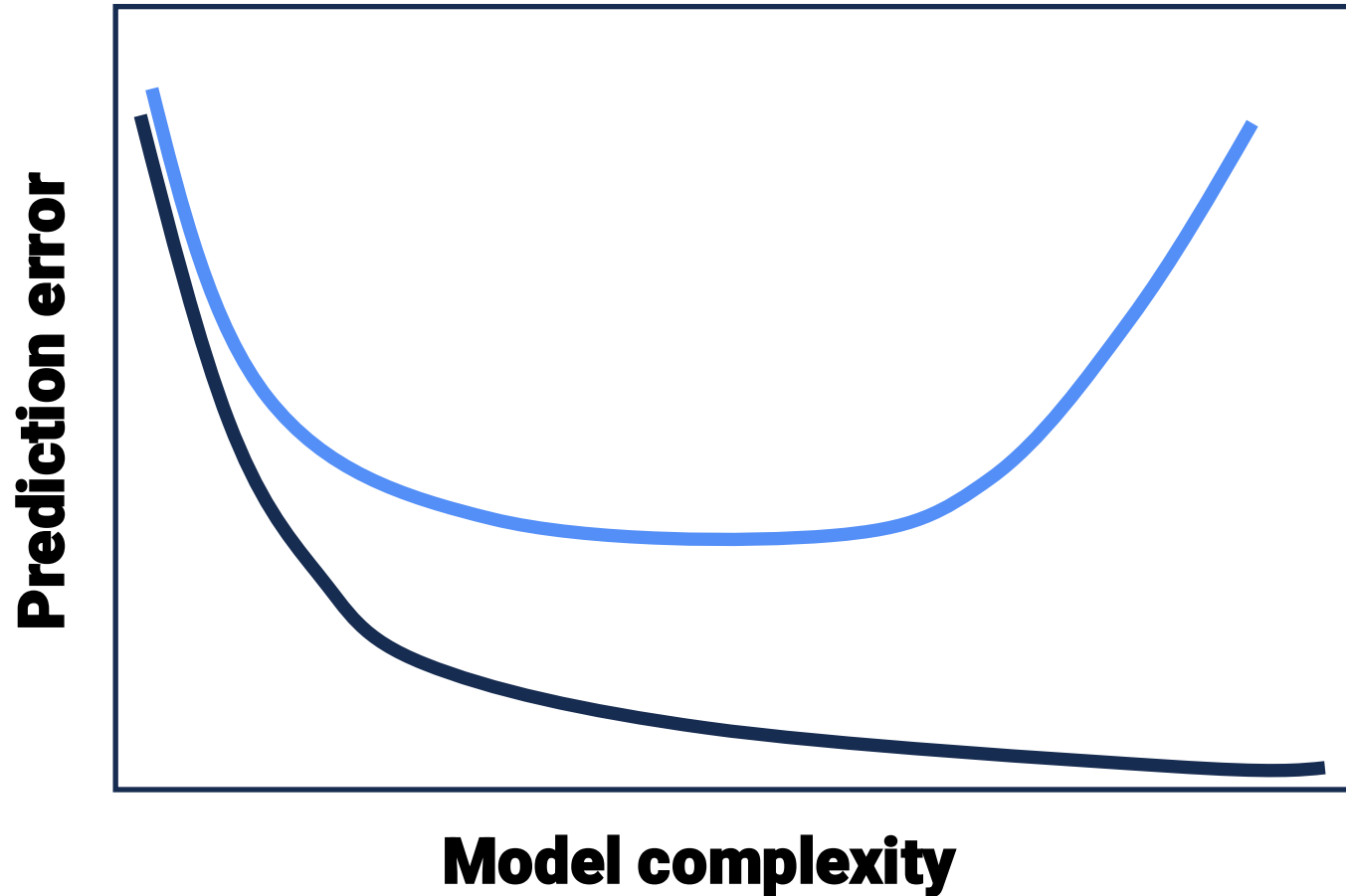
Introduction to Statistical Learning with Applications

CM4: Principal component analysis

Pedro L. C. Rodrigues

Exercise from Carnegie Mellon University (CMU) 11-601 course

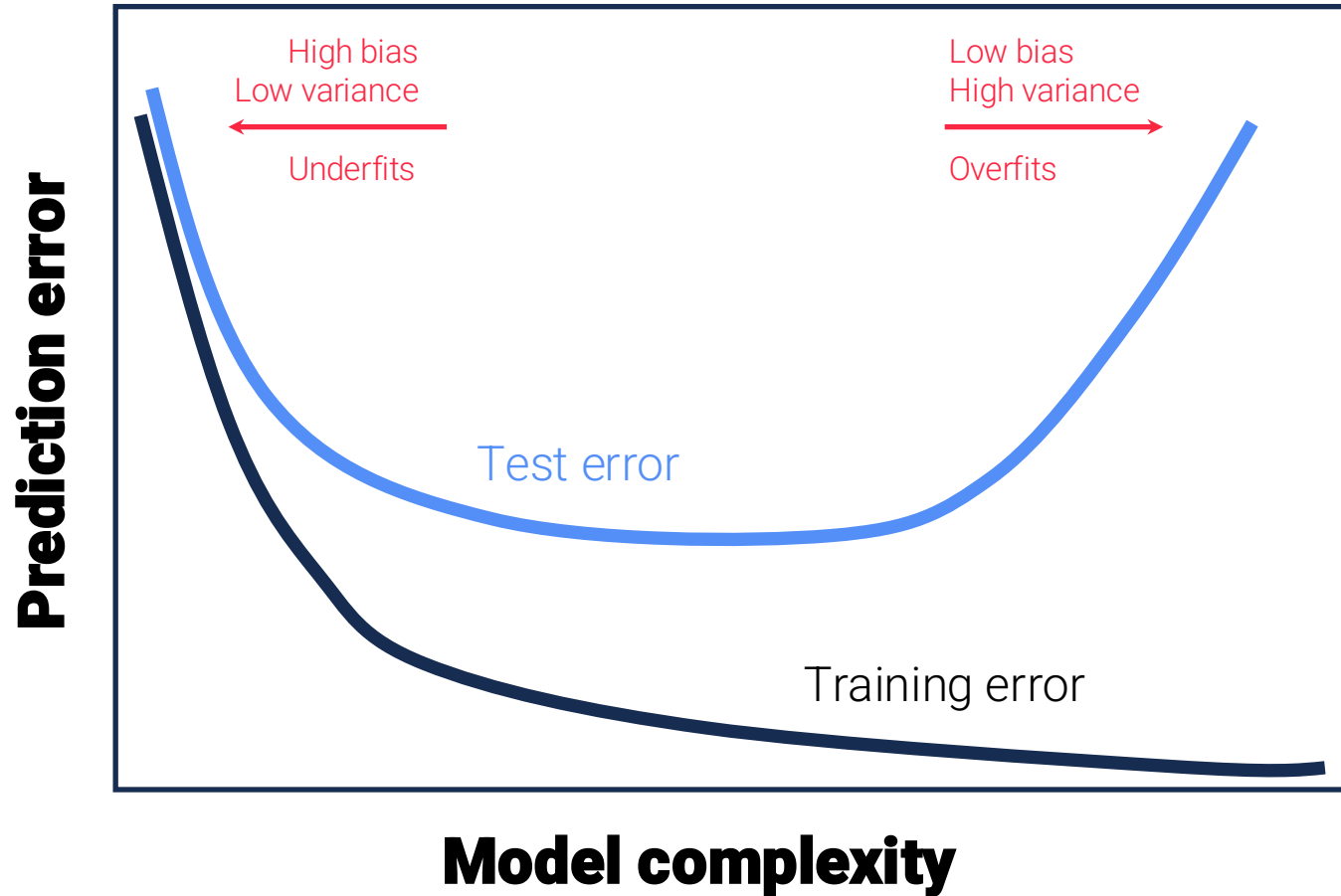
The figure depicts training and validation curves with increasing model complexity



- Which curve is more likely to be the training error ? and the testing error?
- Where in the graph are bias/variance low/high?
- In which regions does the model overfit or underfit?

Exercise from Carnegie Mellon University (CMU) 11-601 course

The figure depicts training and validation curves with increasing model complexity



- Which curve is more likely to be the training error ? and the testing error?
- Where in the graph are bias/variance low/high?
- In which regions does the model overfit or underfit?

Motivation

In TP2 you will be given a dataset with:

- 494 rows representing different individuals
- 5709 columns containing their genetic markers

$$\mathbf{X} \in \mathbb{R}^{494 \times 5709}$$

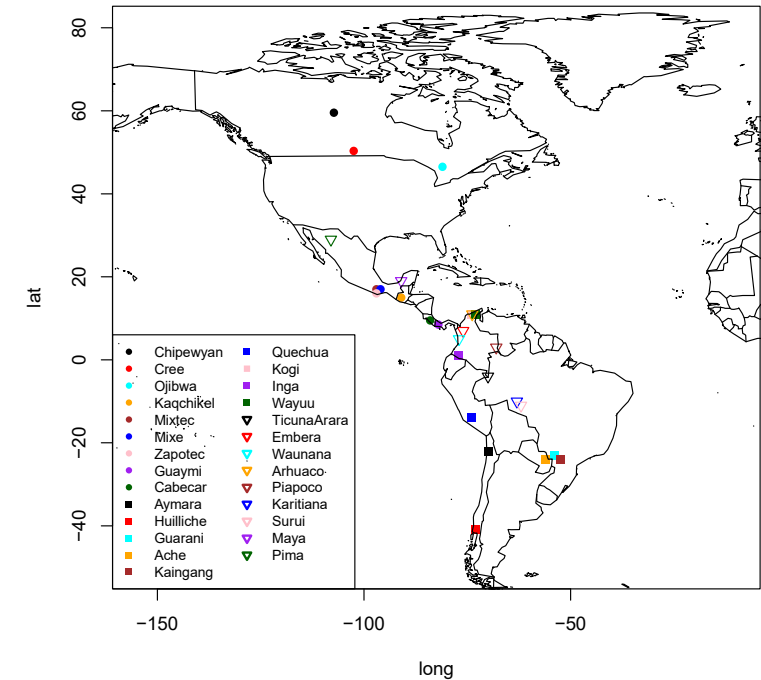
And asked to train a linear regression model that predicts:

- The latitude
- The longitude

$$\mathbf{y}_{\text{lat}}, \mathbf{y}_{\text{long}} \in \mathbb{R}^{494}$$

of the tribe to which each individual belongs.

But $\mathbf{X}^T \mathbf{X} \in \mathbb{R}^{5709 \times 5709}$ is not invertible 🤖 (Why?)

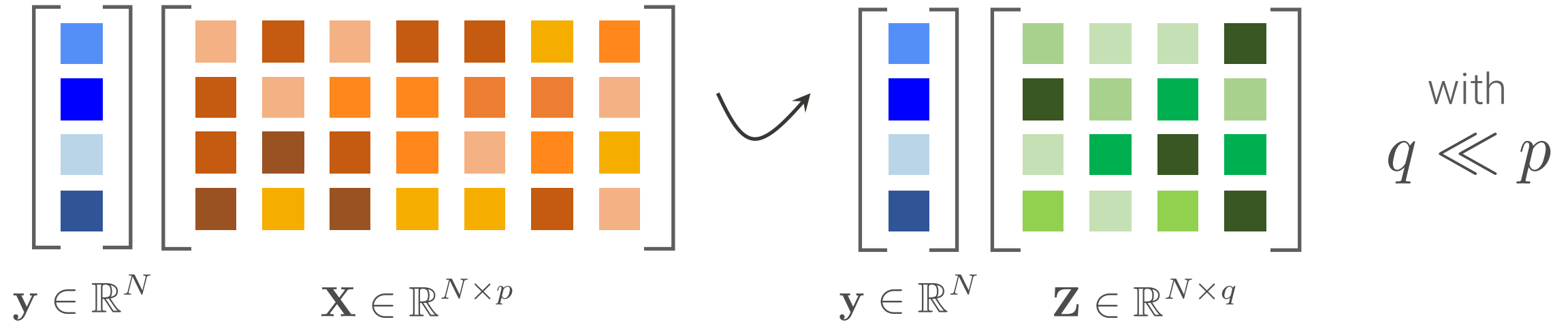


How can we get the coefficients for a linear regression model?

↳ Principal component analysis (PCA) can reduce the dimensionality of the data

Motivation

In other words, we look for a way to transform the data matrix...



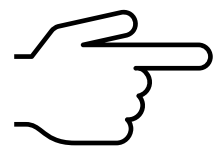
...so that the linear regression model becomes viable and numerically stable

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$



$$\mathbf{y} = \mathbf{Z}\boldsymbol{\gamma} + \boldsymbol{\varepsilon}$$

(We would also like to easily relate the coefficients from each regression model)



- Some important mathematical tools
- Principal Component Analysis (PCA)
- Principal Component Regression (PCR)
- Some practical issues

Important tools from Linear Algebra

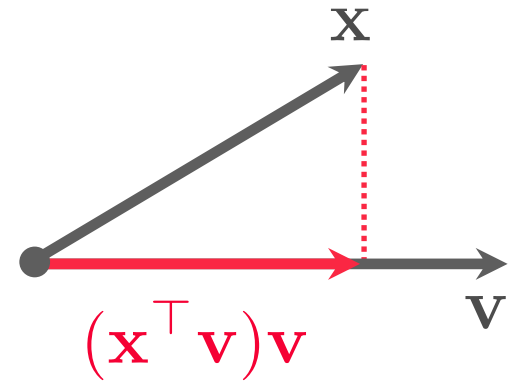
Given two vectors $\mathbf{x}, \mathbf{v} \in \mathbb{R}^p$ with $\|\mathbf{v}\|^2 = \mathbf{v}^\top \mathbf{v} = 1$

- The **projection** of $\mathbf{x}$ onto $\mathbf{v}$ is the vector $(\mathbf{x}^\top \mathbf{v}) \mathbf{v} \in \mathbb{R}^p$

- For data matrix $\mathbf{X} = \begin{bmatrix} x_1^\top \\ \vdots \\ x_N^\top \end{bmatrix} \in \mathbb{R}^{N \times p}$

the projection over $\mathbf{v}$ is $\mathbf{X} \mathbf{v} \mathbf{v}^\top = \begin{bmatrix} (\mathbf{x}_1^\top \mathbf{v}) \mathbf{v}^\top \\ \vdots \\ (\mathbf{x}_N^\top \mathbf{v}) \mathbf{v}^\top \end{bmatrix} \in \mathbb{R}^{N \times p}$

- Two vectors $\mathbf{v}_i, \mathbf{v}_j \in \mathbb{R}^p$ are **orthogonal** if $\mathbf{v}_i^\top \mathbf{v}_j = 0$



Important tools from Linear Algebra

Matrix $\Sigma \in \mathbb{R}^{p \times p}$ is symmetric positive semi-definite (SPSD) matrix if

$$\text{For any } \mathbf{v} \in \mathbb{R}^p, \mathbf{v} \neq \mathbf{0} \text{ we have } \mathbf{v}^\top \Sigma \mathbf{v} \geq 0$$

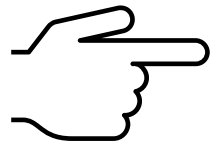
Equivalently, we have that the eigenvalues λ_i and eigenvectors $\mathbf{q}_i$ of Σ are such that

$$\lambda_1 \geq \lambda_2 \geq \dots \lambda_p \geq 0 \text{ and } \mathbf{q}_i^\top \mathbf{q}_j = \delta_{ij}$$

And we can write its eigenvalue decomposition as

$$\Sigma = \begin{bmatrix} | & & | \\ \mathbf{q}_1 & \dots & \mathbf{q}_p \\ | & & | \end{bmatrix} \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_p \end{bmatrix} \begin{bmatrix} - & \mathbf{q}_1^\top & - \\ & \vdots & \\ - & \mathbf{q}_p^\top & - \end{bmatrix} = \sum_{i=1}^p \lambda_i \mathbf{q}_i \mathbf{q}_i^\top$$

- Some important mathematical tools



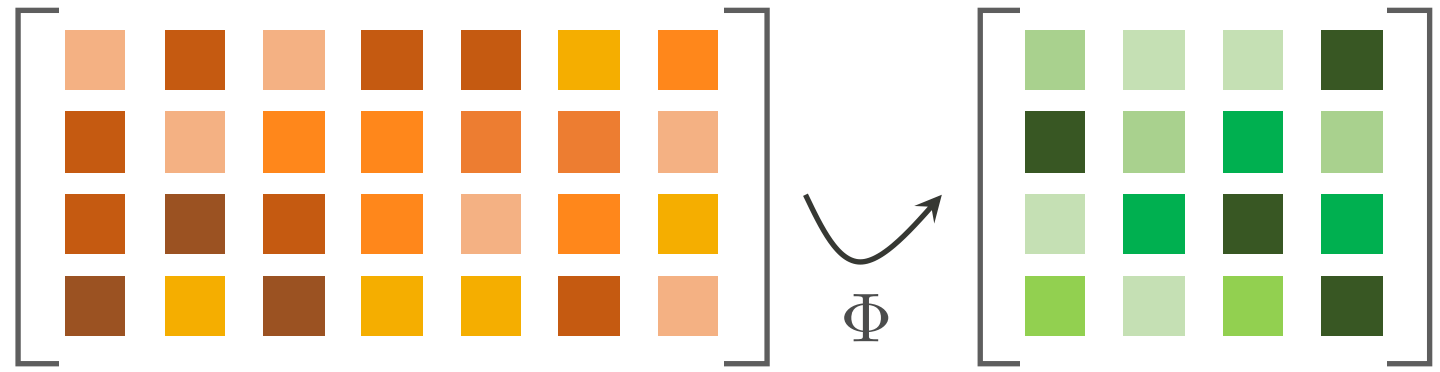
- Principal Component Analysis (PCA)

- Principal Component Regression (PCR)

- Some practical issues

Principal Component Analysis

We want to define a **linear** transformation on our (**zero-mean**) data matrix:



The diagram illustrates a linear transformation Φ from a data matrix $\mathbf{X}$ to a principal component matrix $\mathbf{Z}$. Matrix $\mathbf{X}$ is a 4x7 grid of colored squares (shades of orange, brown, and yellow). Matrix $\mathbf{Z}$ is a 4x4 grid of colored squares (shades of green and dark green). A curved arrow labeled Φ points from $\mathbf{X}$ to $\mathbf{Z}$.

$$\mathbf{X} \in \mathbb{R}^{N \times p}$$
$$\mathbf{Z} \in \mathbb{R}^{N \times q}$$

$$\mathbf{Z}_{N \times q} = \mathbf{X}_{N \times p} \mathbf{W}_{p \times q}$$

Matrix $\mathbf{W}$ should minimize the following **objective** function:

$$\mathcal{L}(\mathbf{W}) = \frac{1}{N} \left\| \begin{matrix} \mathbf{X} \\ N \times p \end{matrix} - \begin{matrix} \mathbf{X} & \mathbf{W} & \mathbf{W}^\top \\ N \times p & p \times q & q \times p \end{matrix} \right\|_2^2 \quad \text{such that} \quad \mathbf{W}^\top \mathbf{W} = \mathbf{I}_q$$

Reconstruction error

Principal Component Analysis

But where does that come from? Let's start from the beginning with $q = 1$

$$\begin{bmatrix} \text{---} & \mathbf{x}_1^\top & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^\top & \text{---} \end{bmatrix} \xrightarrow{\Phi} \begin{bmatrix} z_1 \\ \vdots \\ z_N \end{bmatrix}$$

$\mathbf{X} \in \mathbb{R}^{N \times p} \qquad \mathbf{z} \in \mathbb{R}^N$

Each coordinate of the projection (**score**) is

$$z_i = \mathbf{x}_i^\top \mathbf{w} \quad \text{with} \quad \mathbf{w}^\top \mathbf{w} = 1$$

and we want to minimize **reconstruction error**

$$\sum_{i=1}^N \|\mathbf{x}_i - (\mathbf{x}_i^\top \mathbf{w}) \mathbf{w}\|^2$$

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - (\mathbf{x}_i^\top \mathbf{w}) \mathbf{w}\|^2 = \frac{1}{N} \|\mathbf{X} - \mathbf{X} \mathbf{w} \mathbf{w}^\top\|^2 \quad \text{such that} \quad \mathbf{w}^\top \mathbf{w} = 1$$

How do we **minimize** this objective function?

Principal Component Analysis

How do we **minimize** this objective function? First note that

$$\begin{aligned}\mathcal{L}(\mathbf{w}) &= \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - (\mathbf{x}_i^\top \mathbf{w}) \mathbf{w}\|^2 \\ &= \frac{1}{N} \sum_{i=1}^N \left(\mathbf{x}_i - (\mathbf{x}_i^\top \mathbf{w}) \mathbf{w} \right)^\top \left(\mathbf{x}_i - (\mathbf{x}_i^\top \mathbf{w}) \mathbf{w} \right) \\ &= \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i\|^2 - (\mathbf{w}^\top \mathbf{x}_i)^2\end{aligned}$$

Therefore

$$\underset{\mathbf{w}^\top \mathbf{w}=1}{\text{minimize}} \mathcal{L}(\mathbf{w}) \iff \underset{\mathbf{w}^\top \mathbf{w}=1}{\text{maximize}} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i)^2$$

Principal Component Analysis

We augment the loss function to encode the constraint

$$\underset{\mathbf{w}^\top \mathbf{w} = 1}{\text{maximize}} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i)^2 \iff \underset{\mathbf{w}, \lambda}{\text{maximize}} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i)^2 + \lambda(1 - \mathbf{w}^\top \mathbf{w})$$

then take its gradient and equalize to zero. But first, note that

$$\frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i)^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i^\top \mathbf{w})^2 = \frac{1}{N} \|\mathbf{X}\mathbf{w}\|^2 = \mathbf{w}^\top \left(\frac{1}{N} \mathbf{X}^\top \mathbf{X} \right) \mathbf{w} = \mathbf{w}^\top \Sigma \mathbf{w}$$

covariance of the data
(remember data is zero-mean)

So we have

$$\frac{\partial}{\partial \mathbf{w}} \left(\mathbf{w}^\top \Sigma \mathbf{w} + \lambda(1 - \mathbf{w}^\top \mathbf{w}) \right) = 0 \iff \Sigma \mathbf{w} = \lambda \mathbf{w}$$

Conclusion

$\mathbf{w}$ is an **eigenvector** of the covariance matrix Σ

Principal Component Analysis

Remember Σ is a SPSP matrix, so we should take $\mathbf{w} = \mathbf{q}_1$ since $\lambda_1 \geq \lambda_2 \geq \dots \lambda_p \geq 0$

The actual value of the **reconstruction error** is then

$$\mathcal{L}(\mathbf{q}_1) = \frac{1}{N} \|\mathbf{X}\|^2 - \mathbf{q}_1^\top \Sigma \mathbf{q}_1 = \frac{1}{N} \|\mathbf{X}\|^2 - \lambda_1 = \text{Var}(\mathbf{X}) - \lambda_1$$

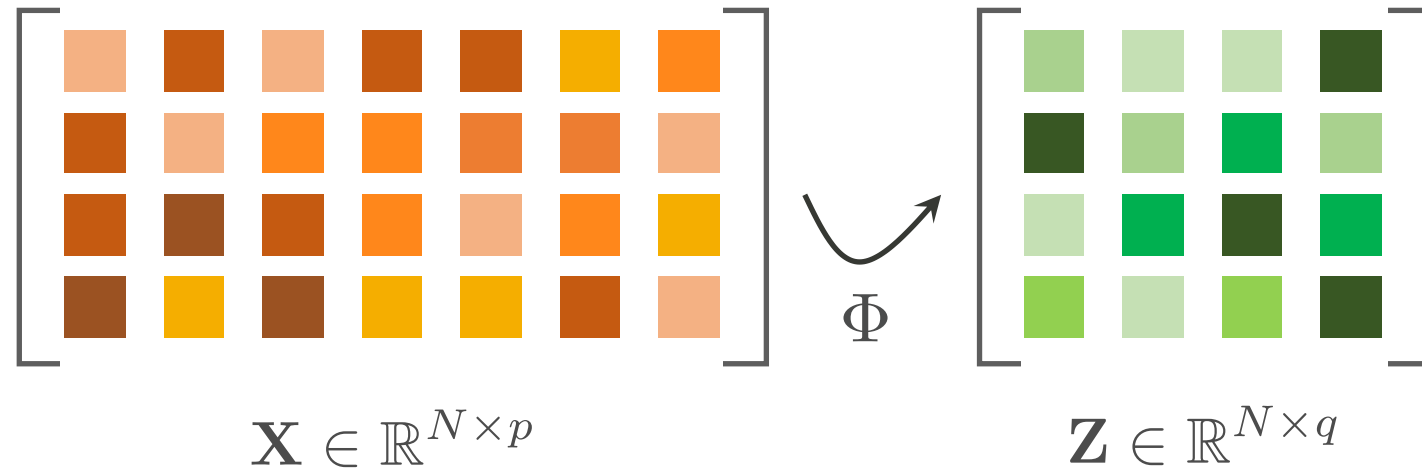
total variance of the data
(remember data is zero-mean)

And the coordinates (aka **scores**) of the projected data points are

$$\mathbf{x}_i \in \mathbb{R}^p \begin{bmatrix} \text{orange square} \\ \text{dark orange square} \\ \text{light orange square} \\ \text{dark orange square} \\ \text{dark orange square} \\ \text{yellow square} \\ \text{orange square} \end{bmatrix} \curvearrowright z_i = \mathbf{x}_i^\top \mathbf{w} \in \mathbb{R} \begin{bmatrix} \text{green square} \end{bmatrix}$$

Principal Component Analysis

What if we want more coordinates to project?



Coming back to the original loss function

$$\mathcal{L}(\mathbf{W}) = \frac{1}{N} \left\| \mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top \right\|_2^2 = \frac{1}{N} \text{tr}(\mathbf{X}^\top \mathbf{X}) - \text{tr}(\mathbf{W}^\top \mathbf{\Sigma} \mathbf{W})$$

and solving in a similar way as before, we get that $\mathbf{W}_q = \begin{bmatrix} | & & | \\ \mathbf{q}_1 & \dots & \mathbf{q}_q \\ | & & | \end{bmatrix}$

Principal Component Analysis

The projected data points are, therefore,

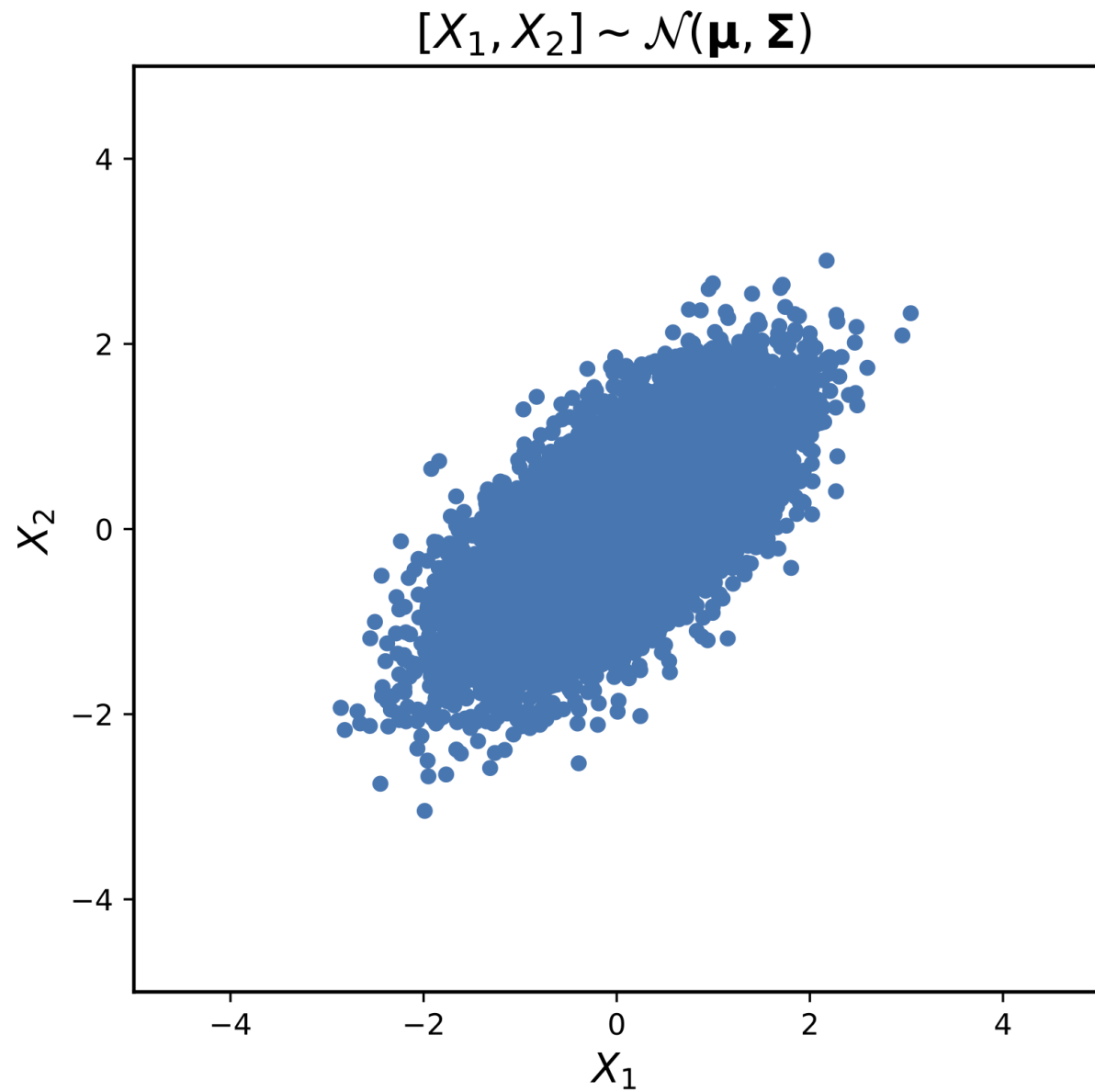
$$\mathbf{Z} = \mathbf{X}\mathbf{W}_q = \begin{bmatrix} \text{---} & \mathbf{x}_1^\top & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^\top & \text{---} \end{bmatrix} \begin{bmatrix} | & & | \\ \mathbf{q}_1 & \dots & \mathbf{q}_q \\ | & & | \end{bmatrix}$$

And the error is $\mathcal{L}(\mathbf{W}_q) = \text{Var}(\mathbf{X}) - \sum_{i=1}^q \lambda_i = \underbrace{\sum_{j=1}^p \lambda_j}_{\text{why?}} - \sum_{i=1}^q \lambda_i = \sum_{i=q+1}^p \lambda_i$

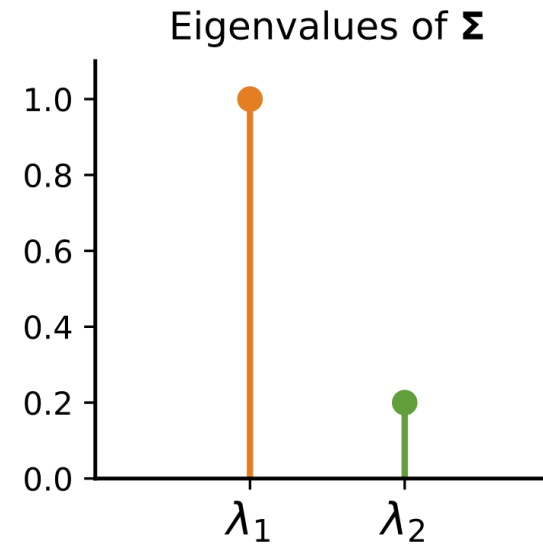
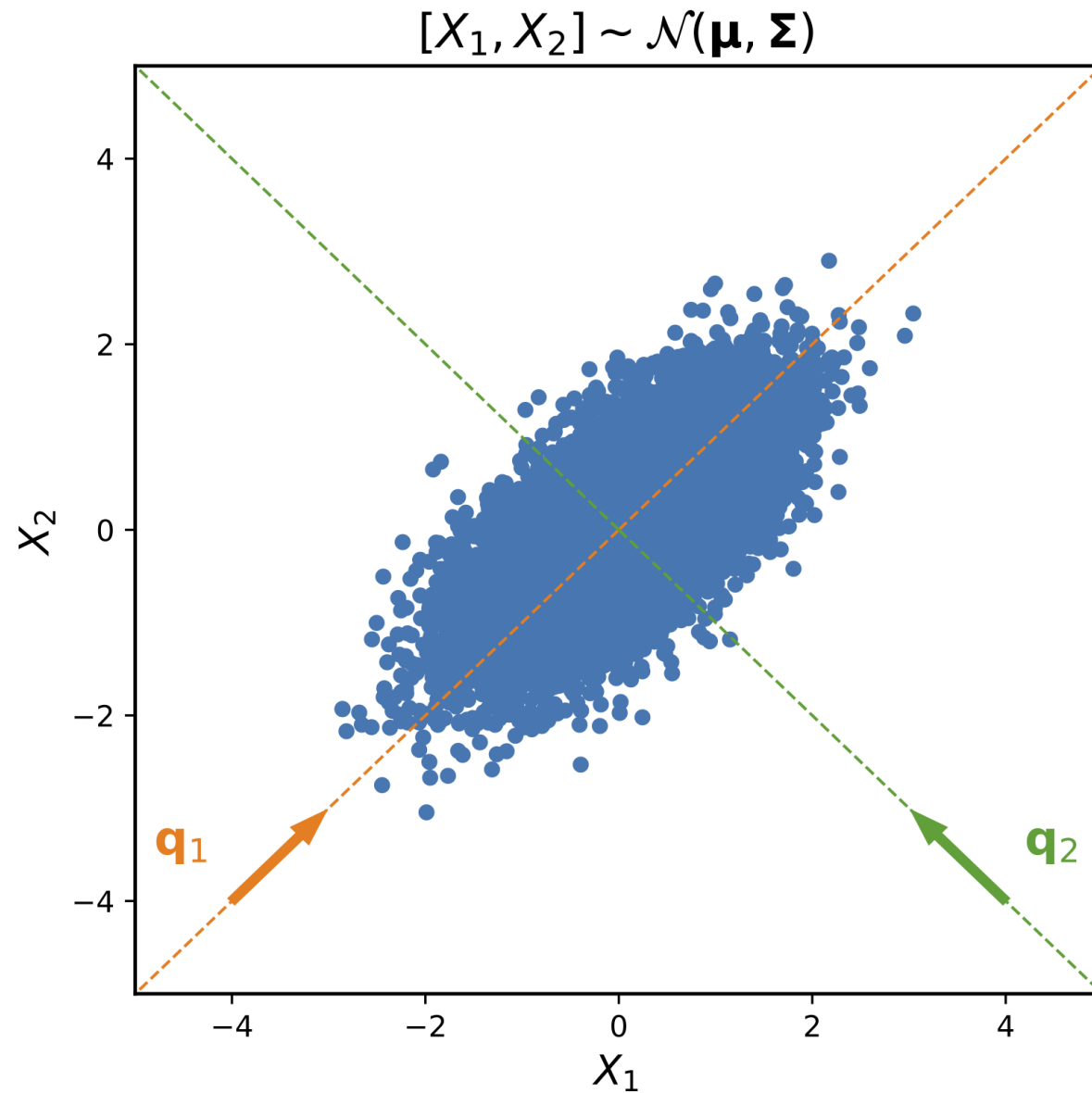
What would be a good choice for q ?

We may want to control the error as in $\mathcal{L}(\mathbf{W}_q) \leq \varepsilon \text{Var}(\mathbf{X}) \iff \frac{\sum_{i=1}^q \lambda_i}{\sum_{j=1}^p \lambda_j} \geq 1 - \varepsilon$

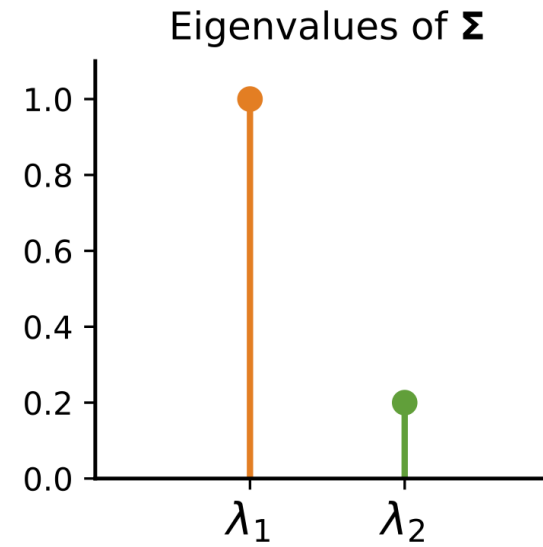
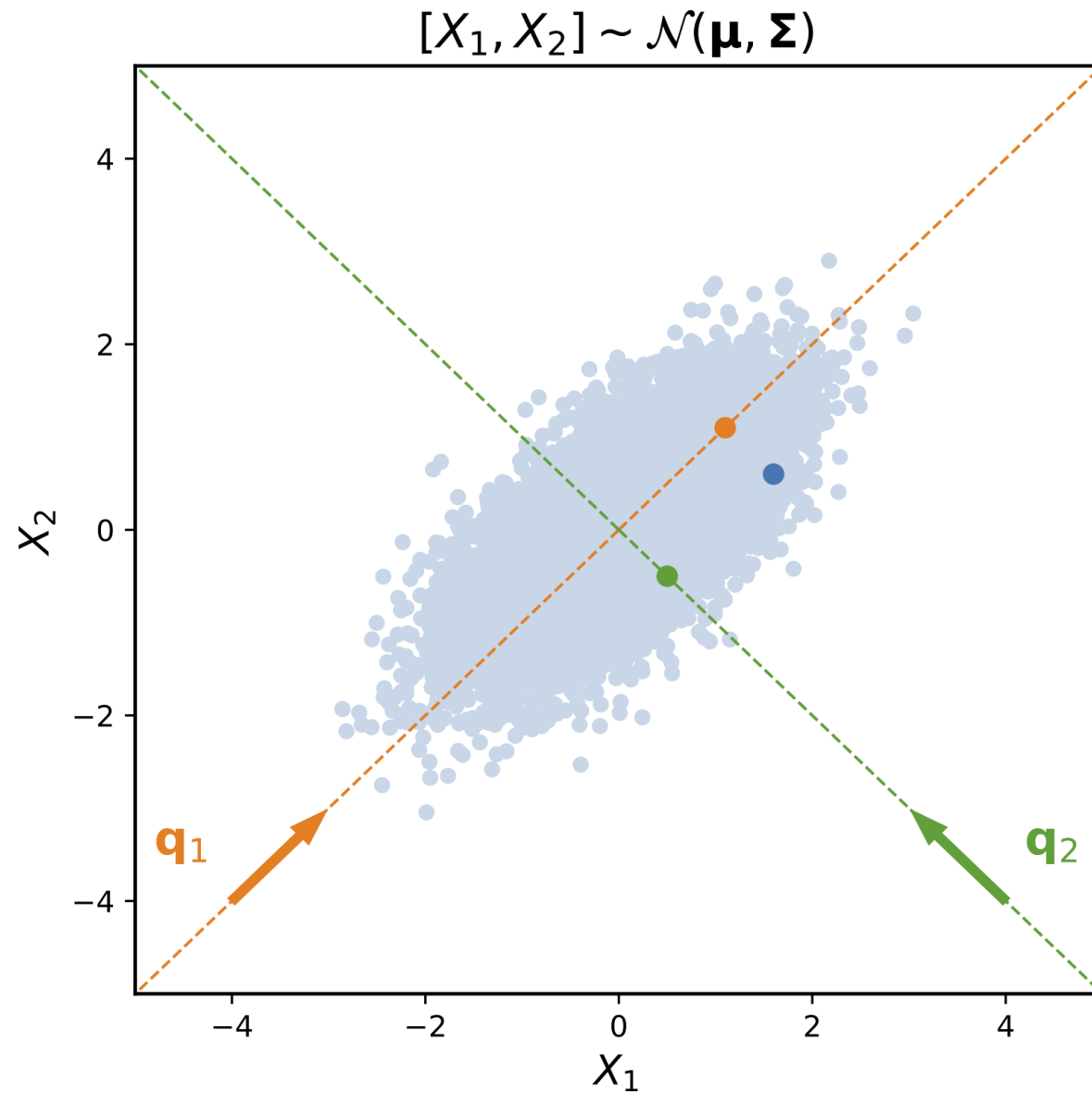
Principal Component Analysis



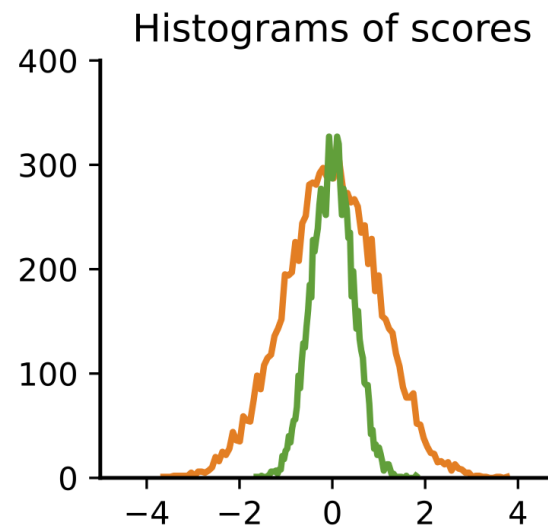
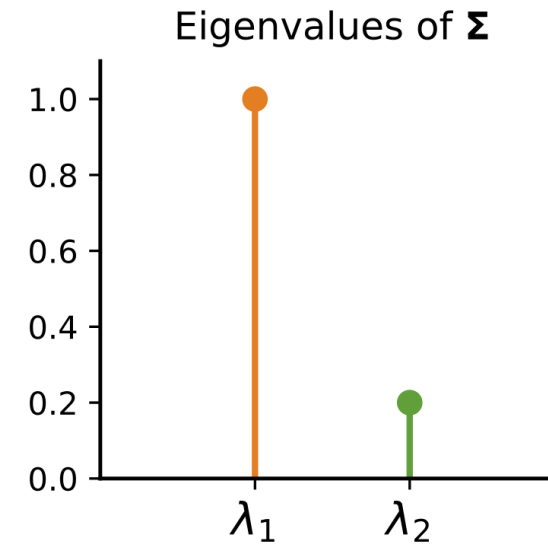
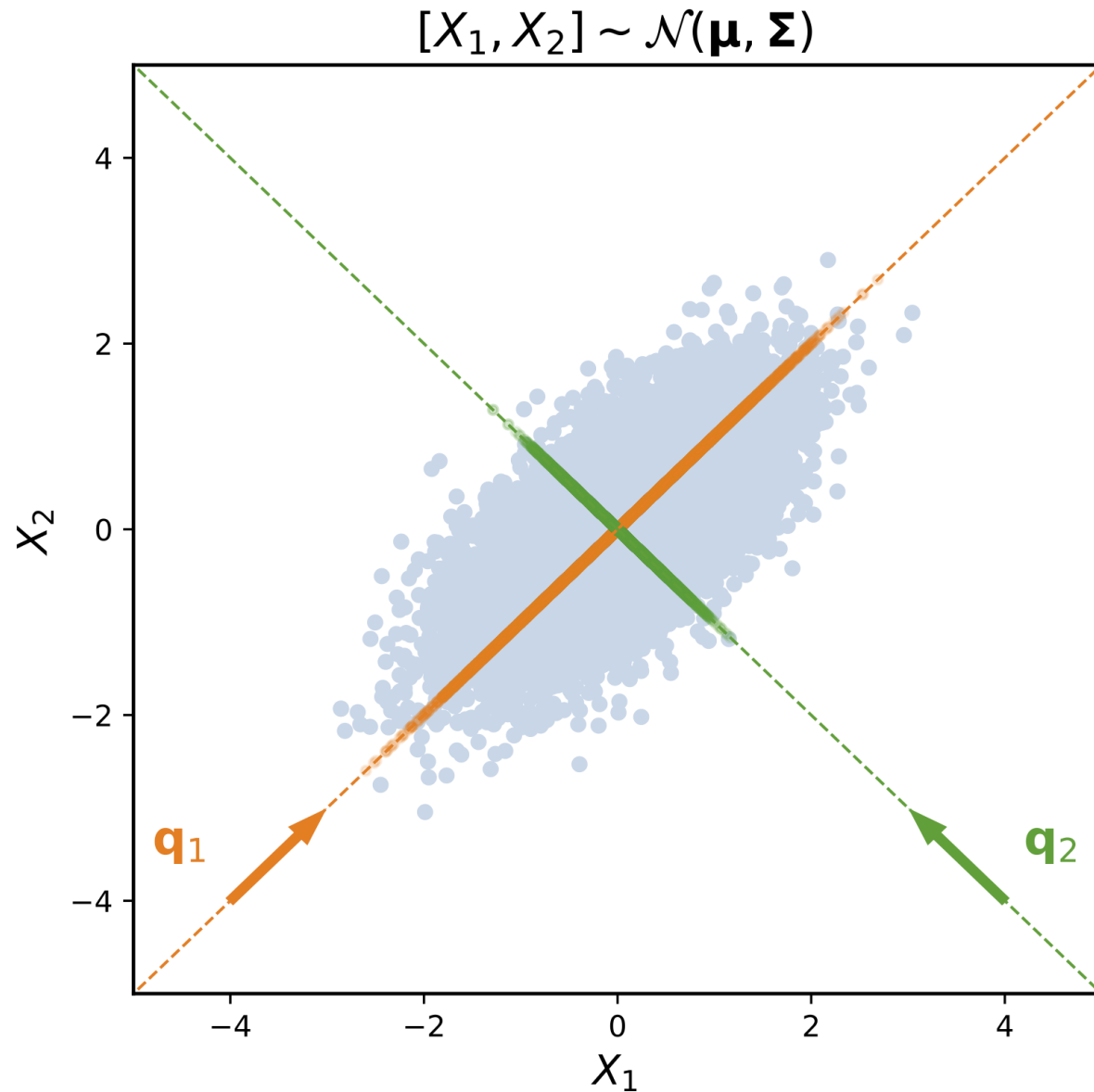
Principal Component Analysis



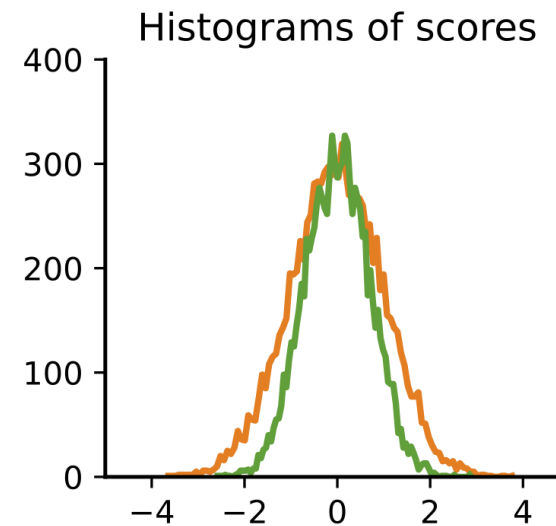
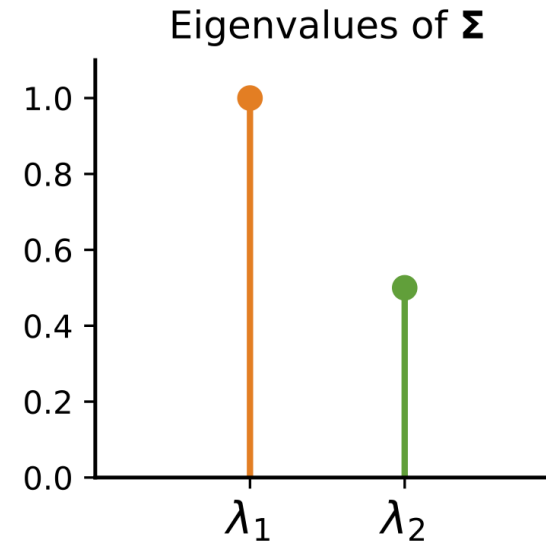
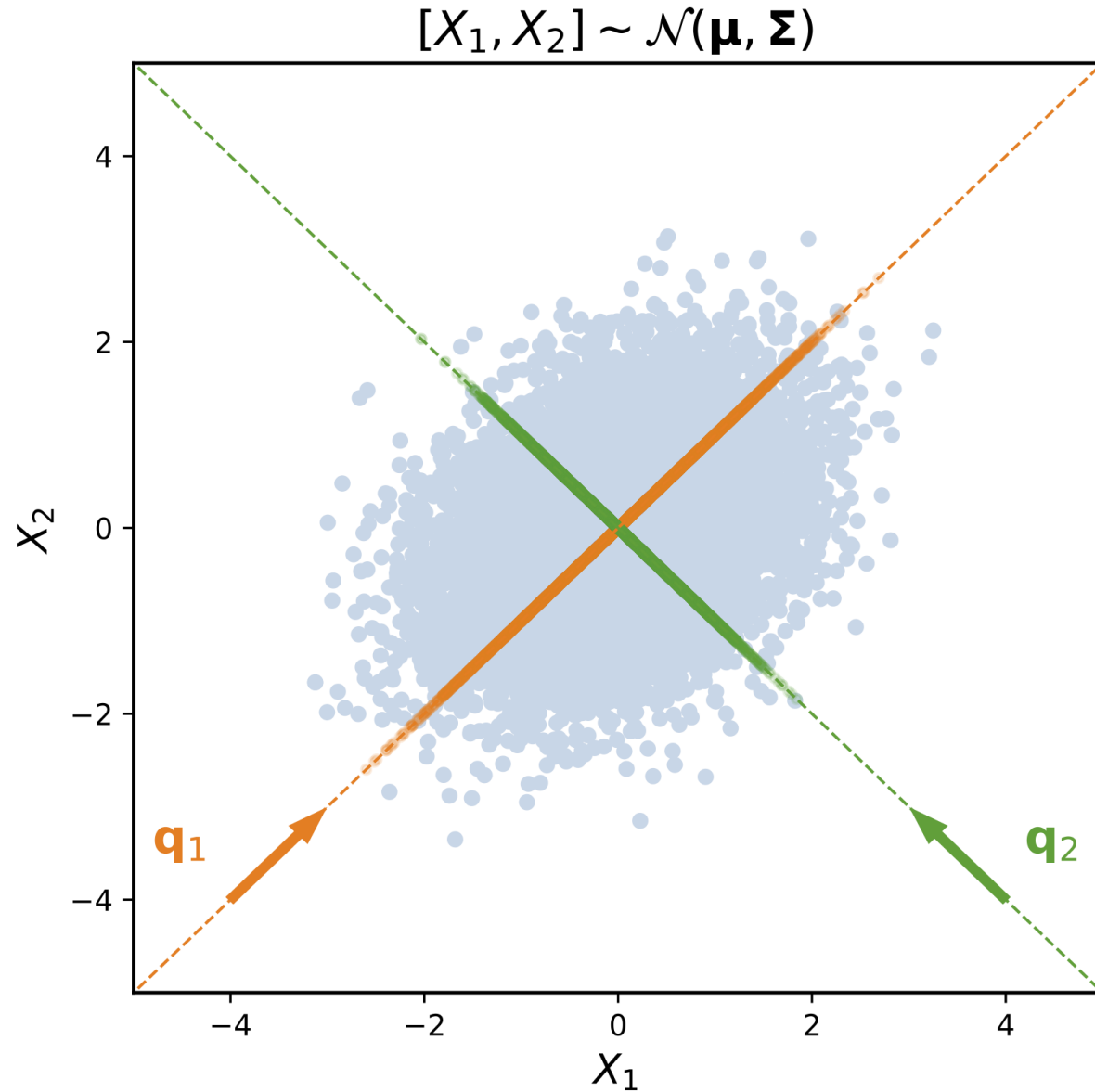
Principal Component Analysis



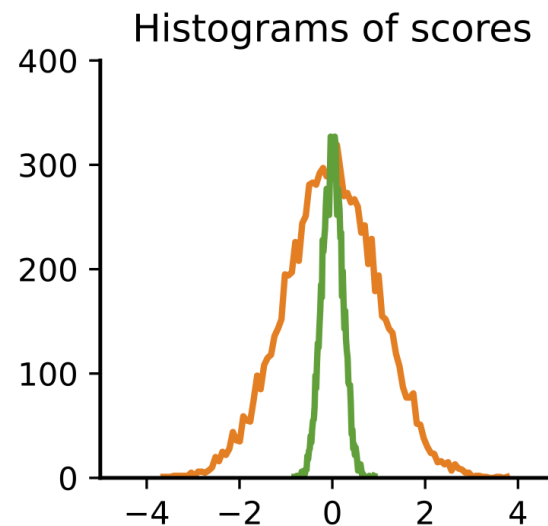
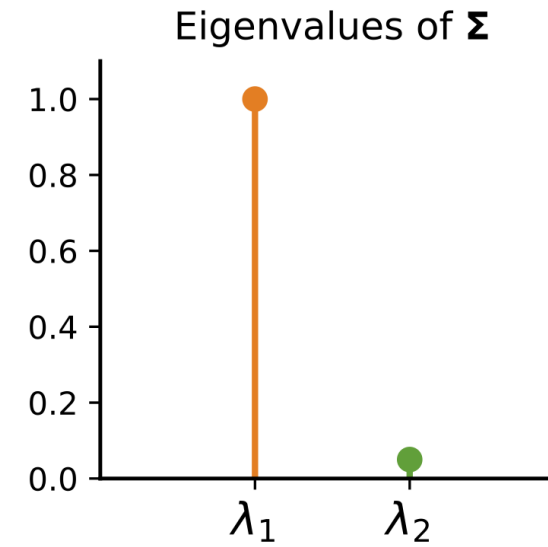
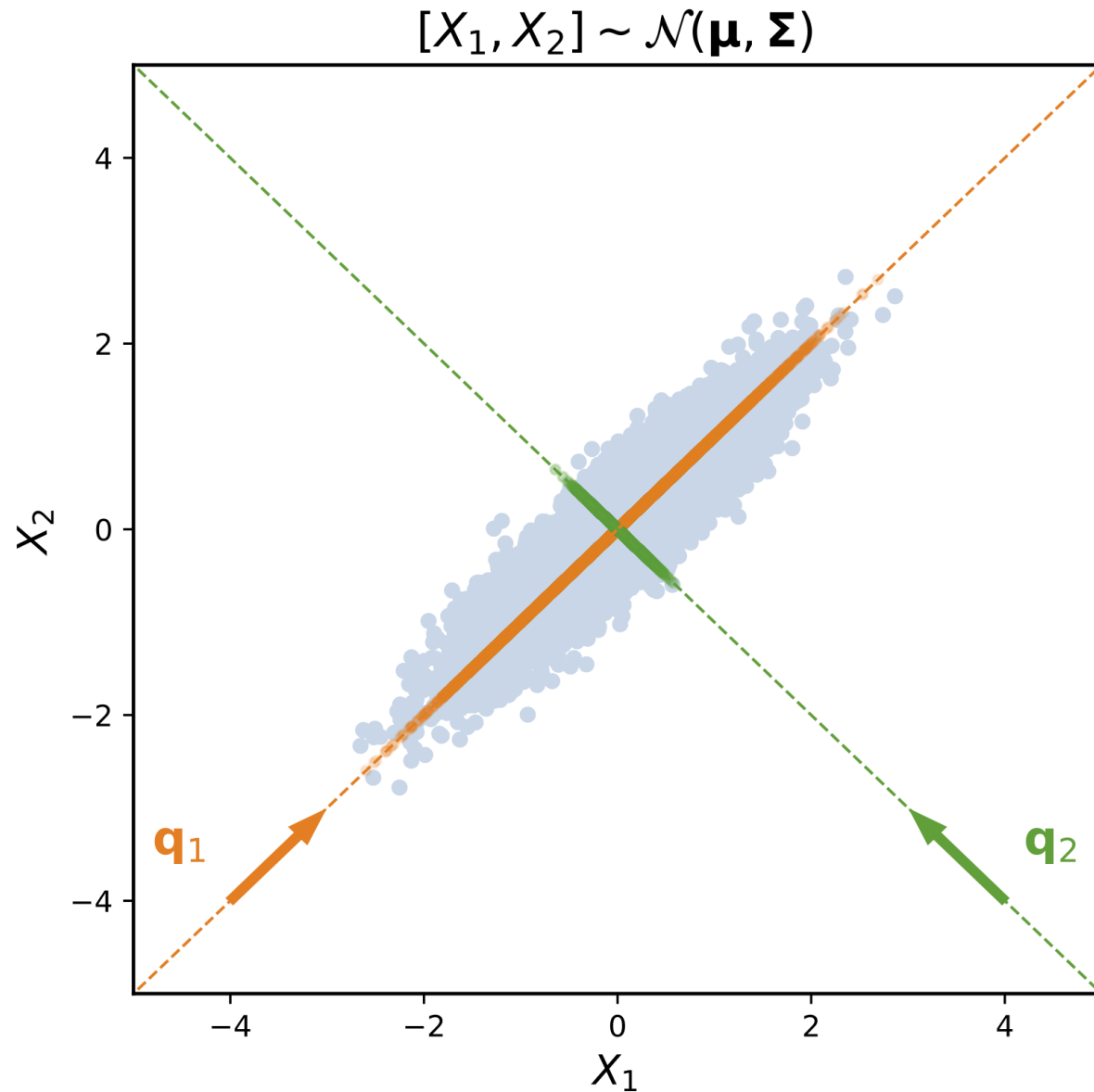
Principal Component Analysis



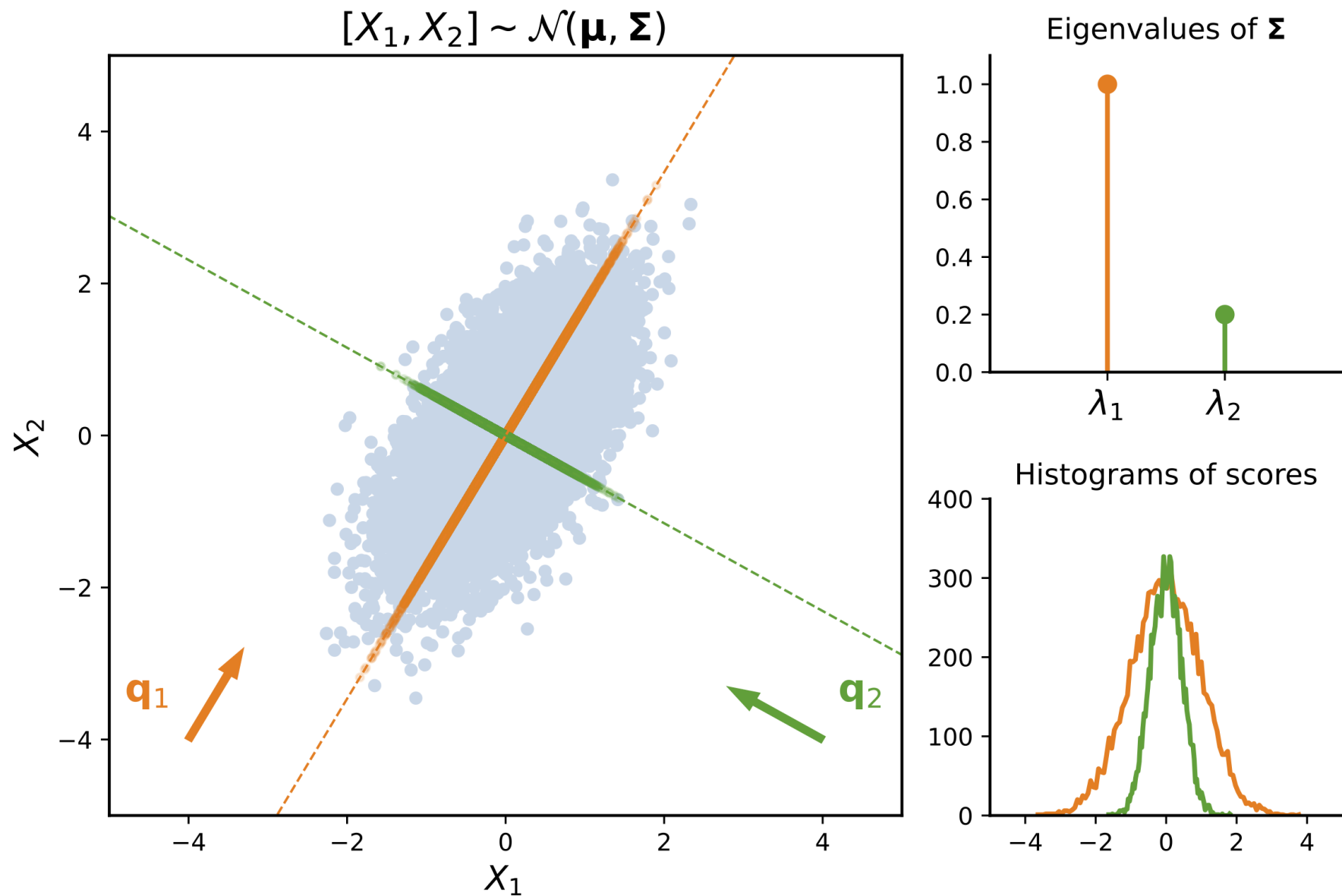
Principal Component Analysis



Principal Component Analysis



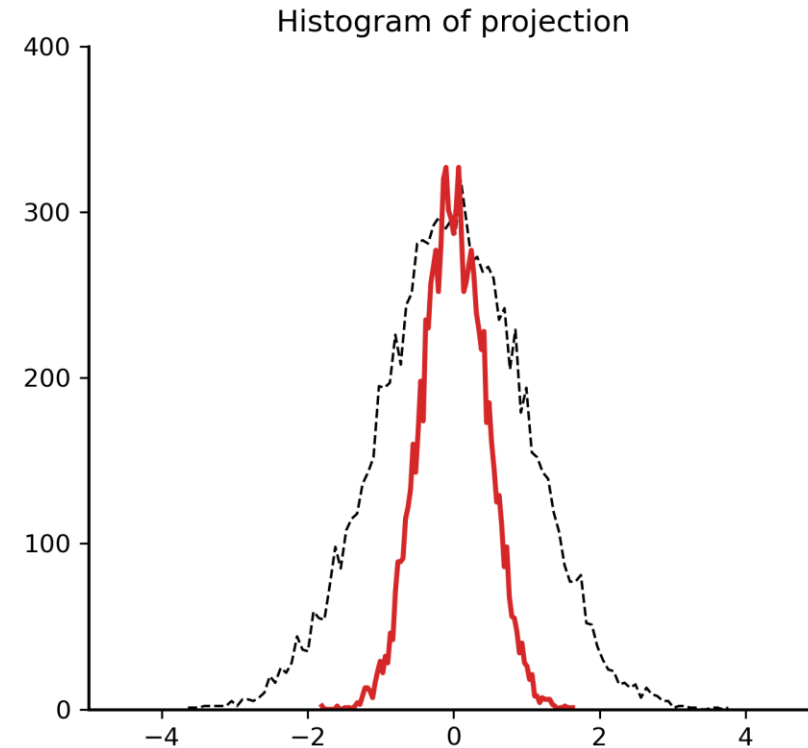
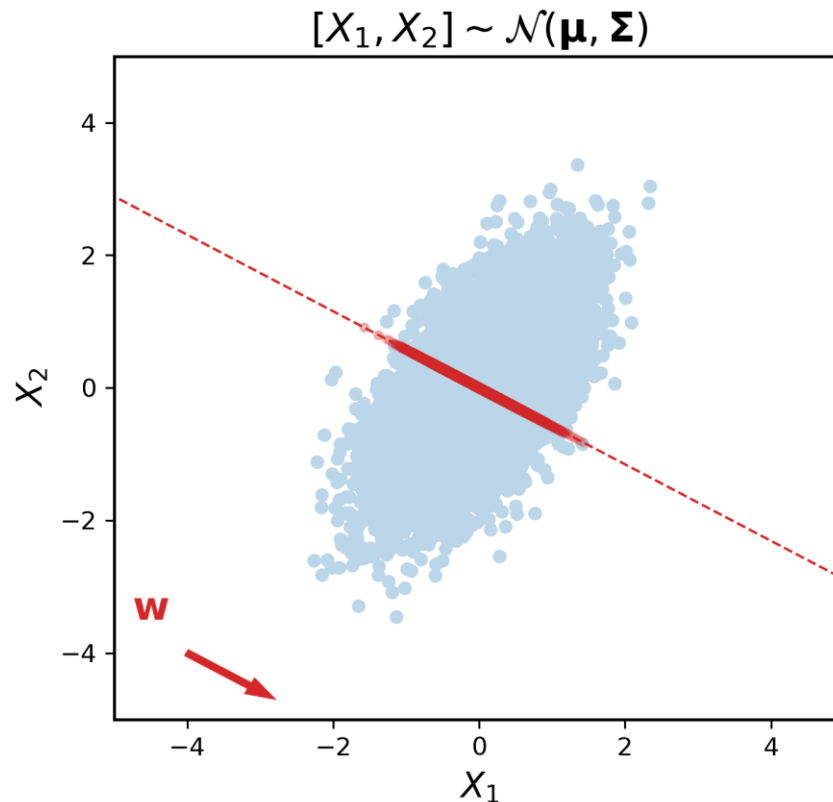
Principal Component Analysis



Principal Component Analysis

Minimize reconstruction error is the same **maximizing the variance** of the projections

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i)^2 = \mathbb{E}[(\mathbf{w}^\top \mathbf{x})^2] = \text{Var}(\mathbf{w}^\top \mathbf{x}) \quad \text{remember data is zero-mean}$$

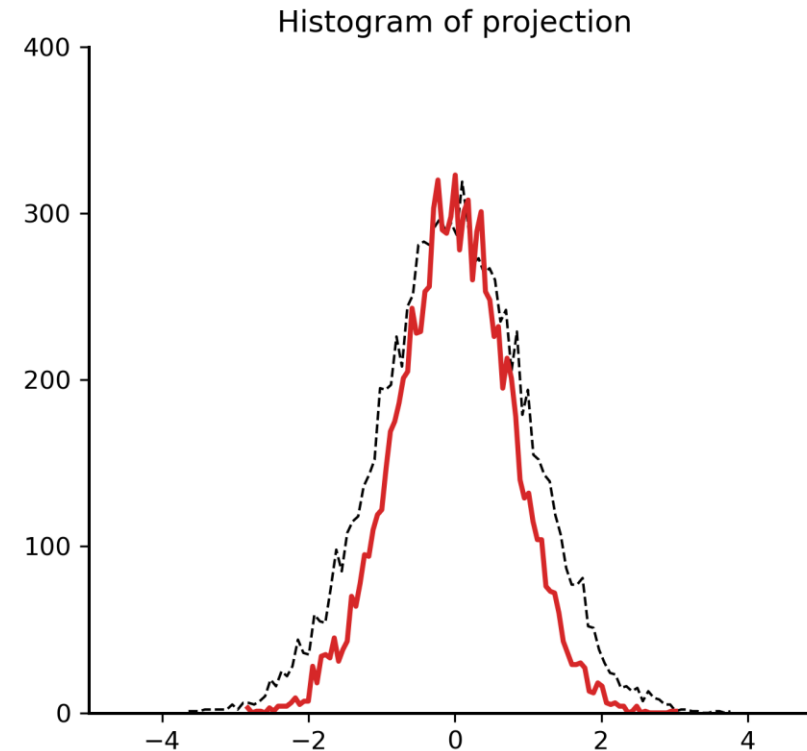
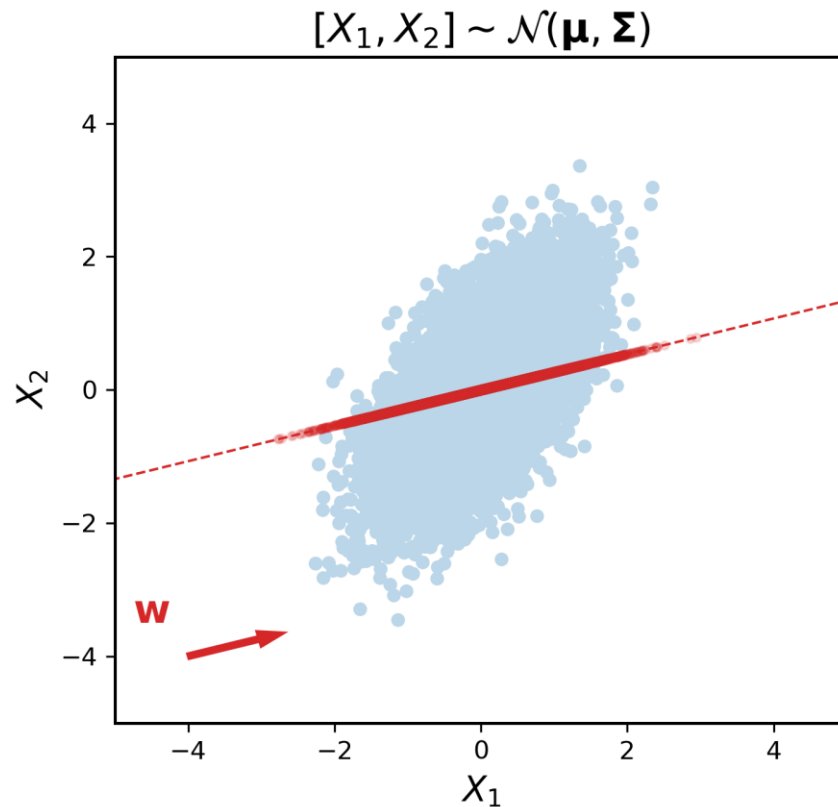


Principal Component Analysis

Minimize reconstruction error is the same **maximizing the variance** of the projections

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i)^2 = \mathbb{E}[(\mathbf{w}^\top \mathbf{x})^2] = \text{Var}(\mathbf{w}^\top \mathbf{x})$$

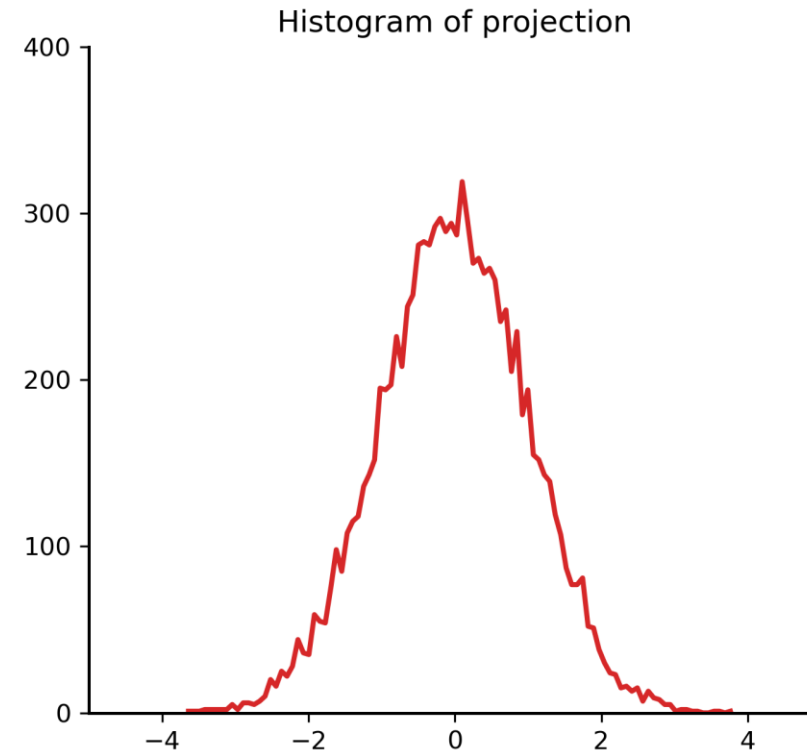
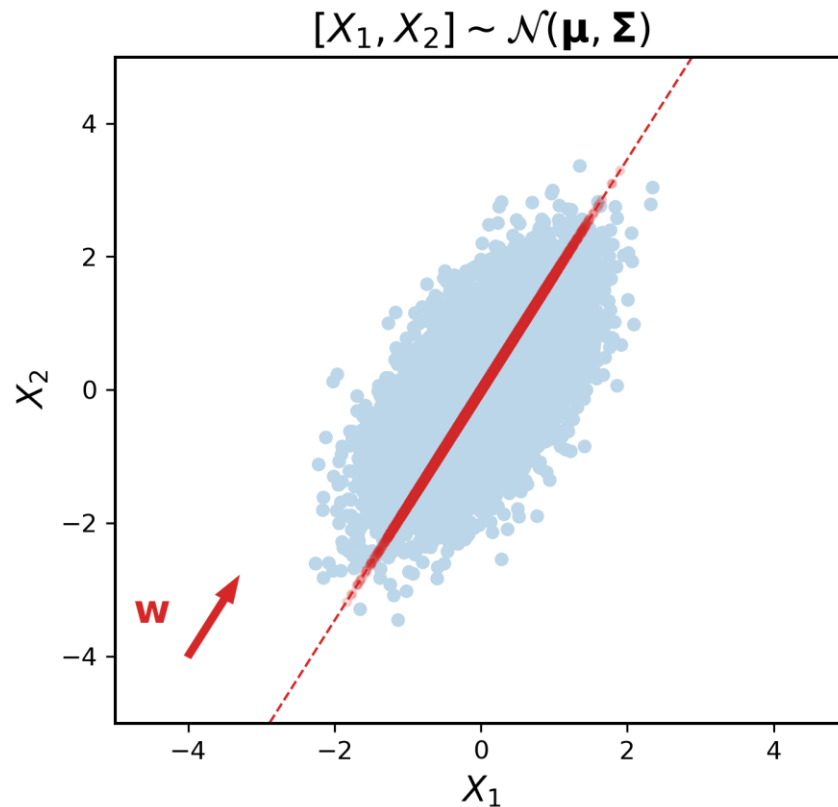
remember data is zero-mean



Principal Component Analysis

Minimize reconstruction error is the same **maximizing the variance** of the projections

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i)^2 = \mathbb{E}[(\mathbf{w}^\top \mathbf{x})^2] = \text{Var}(\mathbf{w}^\top \mathbf{x}) \quad \text{remember data is zero-mean}$$



Principal Component Analysis

PCA is often used as a tool for [visualizing](#) data with many predictors.

The “Economics” dataset describes 12 countries with 6 economic indicators from 1991

observations

features

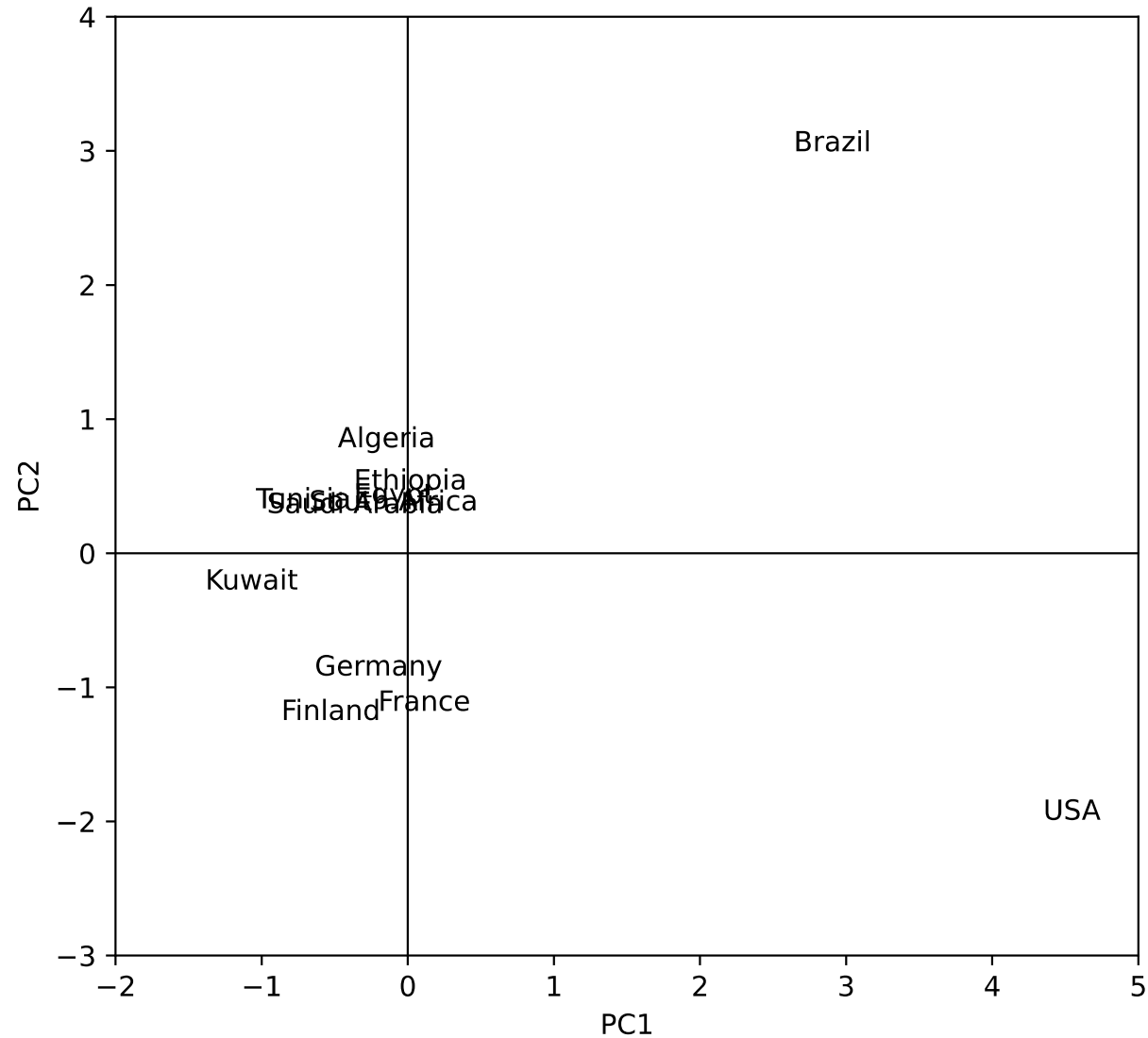
▸ In [3]: df

Out[3]:

	GNB.capita	inflation	unemployment	external.trade	population	area
0	2810	14.7	0.0	6.3	36.8	1.22
1	1540	50.0	24.3	4.2	26.1	2.38
2	24130	3.5	5.1	23.5	81.0	0.35
3	7328	4.4	0.0	22.1	14.8	2.15
4	2400	440.8	4.8	10.5	153.0	8.51
5	620	19.8	17.5	-5.9	56.1	1.00
6	21890	4.2	6.7	-73.4	255.0	9.36
7	110	35.7	0.0	-0.6	54.6	1.22
8	25800	4.1	7.7	2.2	5.0	0.33
9	21030	3.2	9.4	-10.1	57.2	0.55
10	14000	3.3	0.0	20.0	1.3	0.02
11	1350	8.2	15.0	-0.9	8.6	0.16

Principal Component Analysis

We can plot on a 2D plane the coordinates of the projections on the first two PC



Principal Component Analysis

This can be **very easily** done using scikit-learn



[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#)



sklearn.datasets

sklearn.decomposition

DictionaryLearning

FactorAnalysis

FastICA

IncrementalPCA

KernelPCA

LatentDirichletAllocation

MiniBatchDictionaryLearning

MiniBatchNMF

MiniBatchSparsePCA

NMF

PCA

SparseCoder

[Home](#) > [API Reference](#) > [sklearn.decomposition](#) > [PCA](#)

PCA

```
class sklearn.decomposition.PCA(n_components=None, *, copy=True,  
whiten=False, svd_solver='auto', tol=0.0, iterated_power='auto',  
n_oversamples=10, power_iteration_normalizer='auto', random_state=None)
```

Principal component analysis (PCA).

[\[source\]](#)

Linear dimensionality reduction using Singular Value Decomposition of the data to project it to a lower dimensional space. The input data is centered but not scaled for each feature before applying the SVD.

It uses the LAPACK implementation of the full SVD or a randomized truncated SVD by the method of Halko et al. 2009, depending on the shape of the input data and the number of components to extract.

Principal Component Analysis

This can be **very easily** done using scikit-learn

```
X = df.values
pca = PCA(n_components=2)
pca.fit(X)
Z = pca.transform(X)
```

We can also get the values of the **projection matrix** for the PCA

◦ In [51]: `pca.components_`

Out [51]:

```
array([[ 0.09086083,  0.29040994, -0.01546447, -0.45931569,  0.59388598,
         0.58608511],
       [-0.63680828,  0.63938098,  0.04793568,  0.38336688, -0.08569851,
         0.1704551 ]])
```

In [52]: `pca.components_.shape`

Out [52]: (2, 6)

Principal Component Analysis

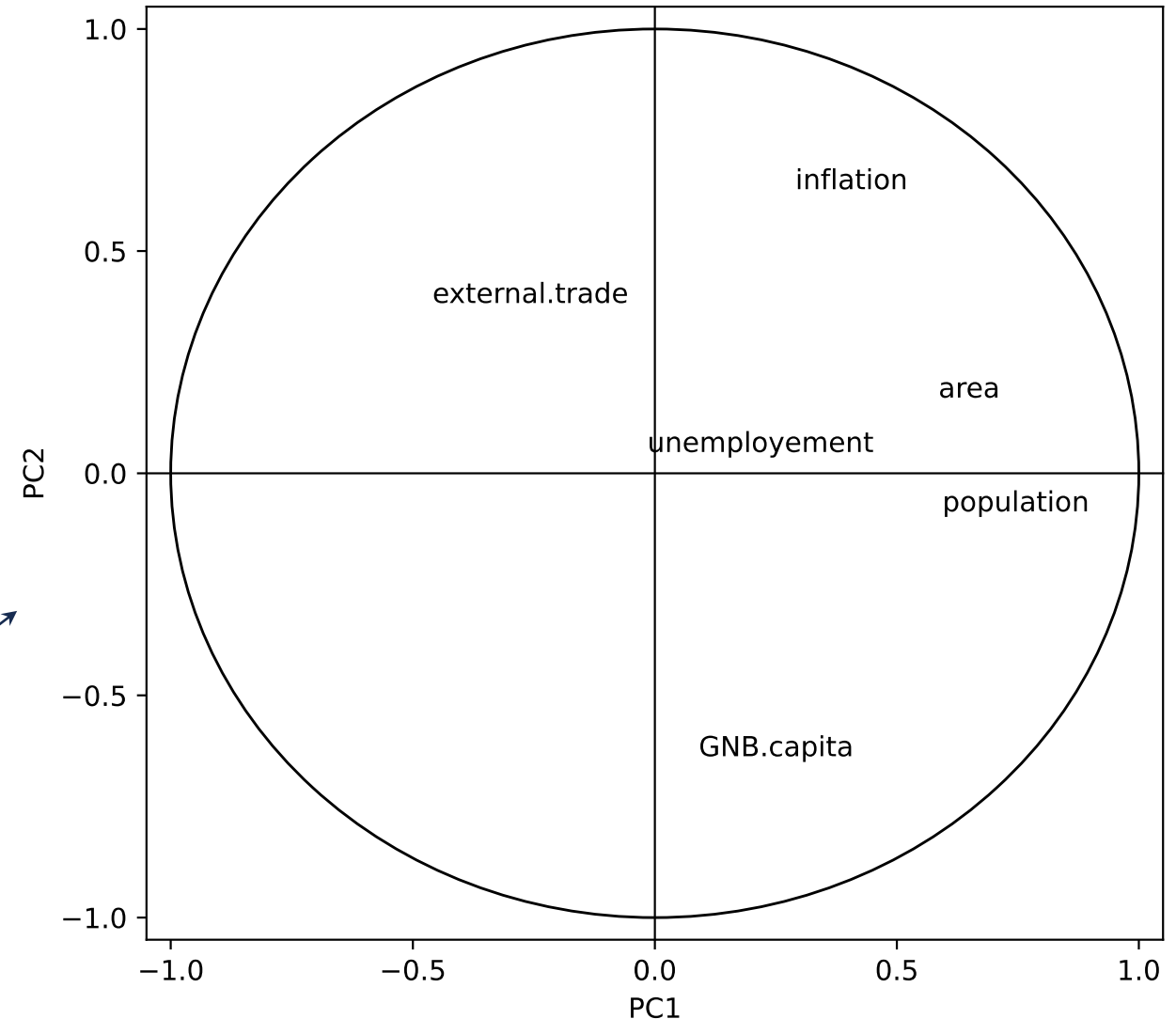
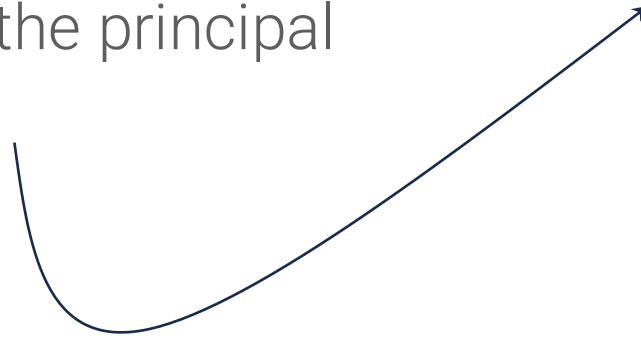
Remember that the components are unit vectors, i.e. directions

```
In [67]: W = pca.components_
```

```
In [68]: np.linalg.norm( W[0] )  
Out[68]: 0.9999999999999999
```

```
In [69]: np.linalg.norm( W[1] )  
Out[69]: 0.9999999999999999
```

We can inspect how each feature decomposes along the principal components



Principal Component Analysis

Remember that the components are unit vectors, i.e. directions

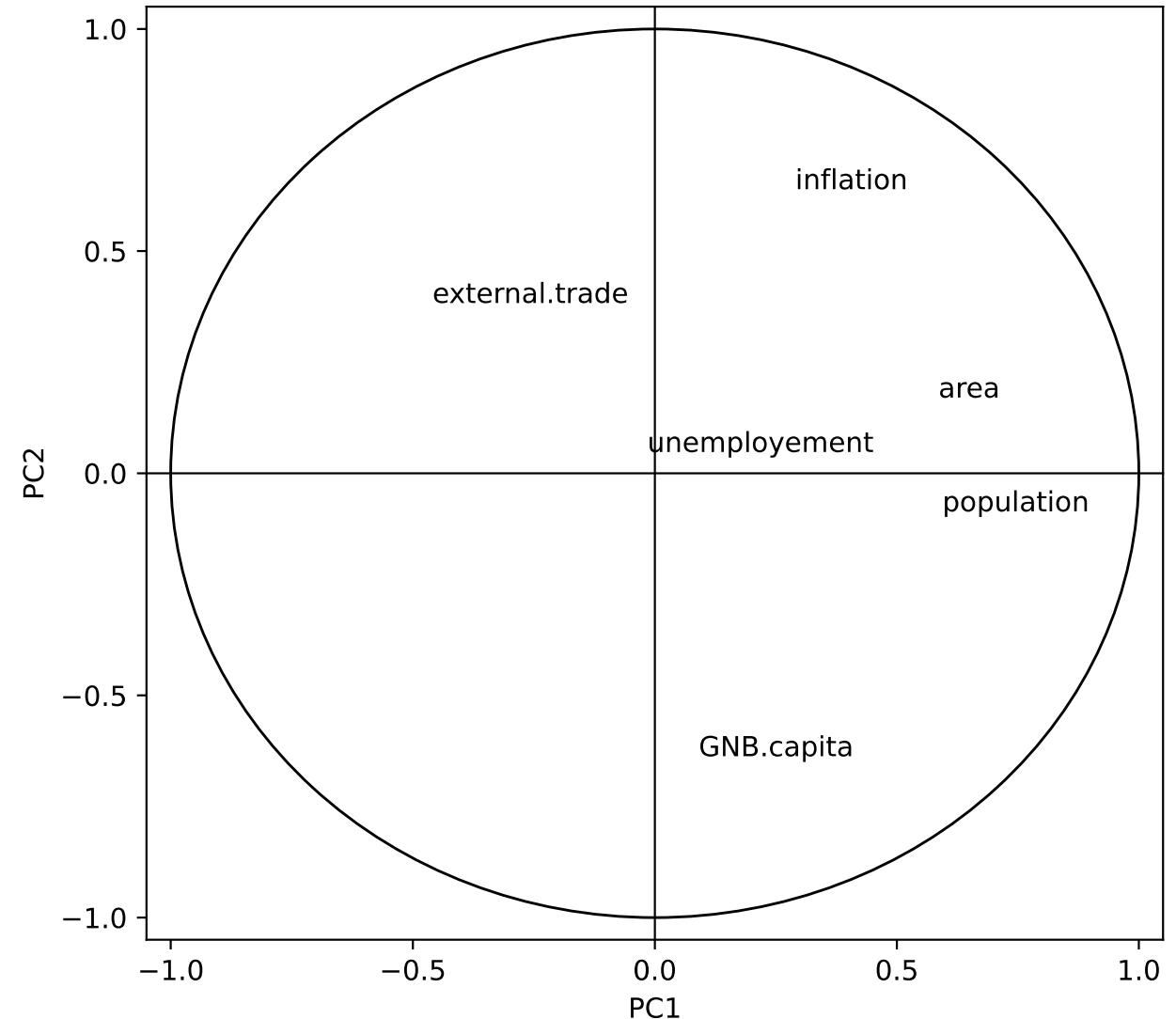
```
In [67]: W = pca.components_
```

```
In [68]: np.linalg.norm( W[0] )  
Out[68]: 0.9999999999999999
```

```
In [69]: np.linalg.norm( W[1] )  
Out[69]: 0.9999999999999999
```

We can inspect how each feature decomposes along the principal components

However, matrix **W** only conveys direction information. How to include the importance of each PC?



Principal Component Analysis : the correlation plot

It is common to define the notion of **loadings** of the PCA as per

$$\ell_{ij} = \mathbf{W}_{ij} \sqrt{\frac{\text{Var}(Z_j)}{\text{Var}(X_i)}} = \mathbf{W}_{ij} \sqrt{\frac{\lambda_j}{\text{Var}(X_i)}}$$

which describes how much the variable **i** is **correlated** to the principal component **j**

$$\mathbf{W} = \left[\begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \right] \quad \left. \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \\ \\ \end{array} \right\} p \text{ variables}$$

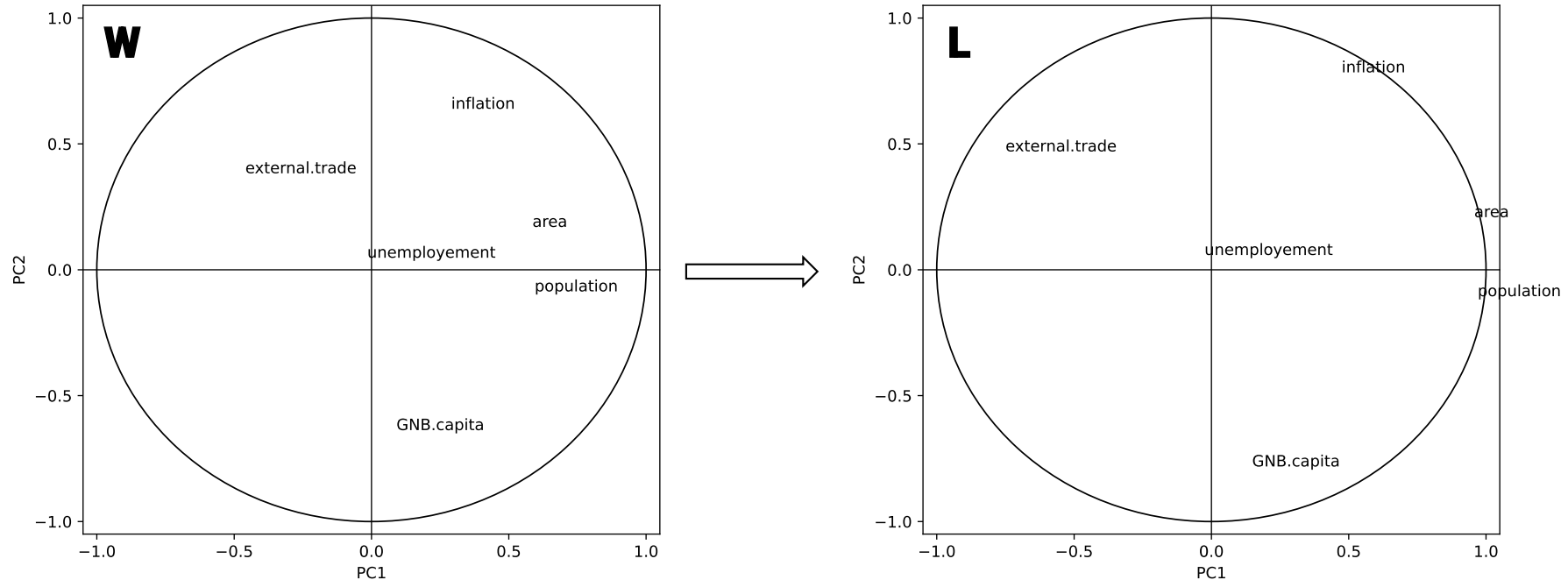
$\underbrace{\hspace{10em}}_{q \text{ principal components}}$

Principal Component Analysis : the correlation plot

It is common to define the notion of **loadings** of the PCA as per

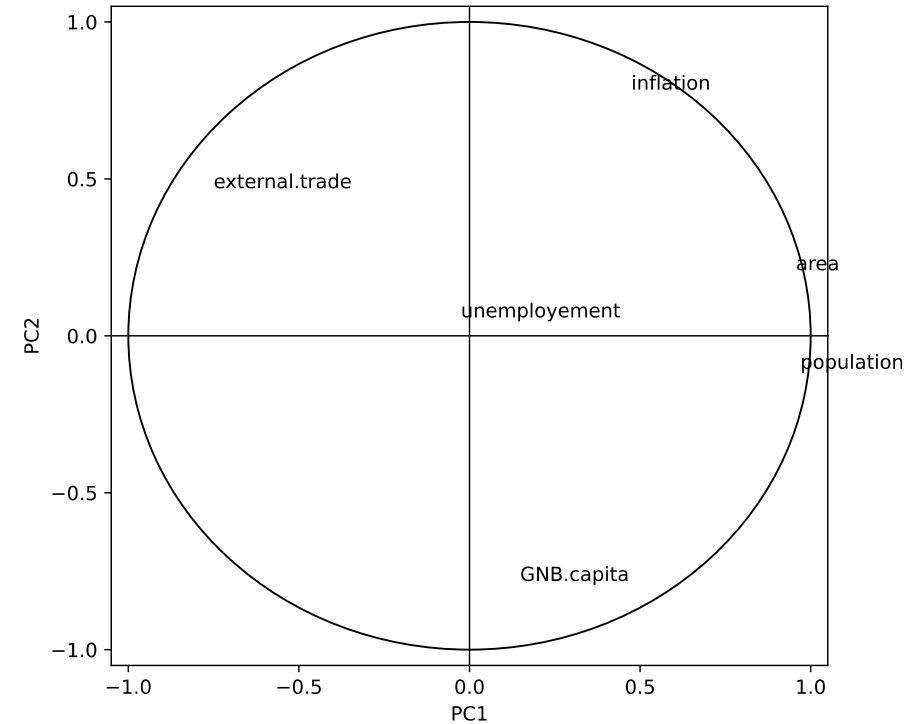
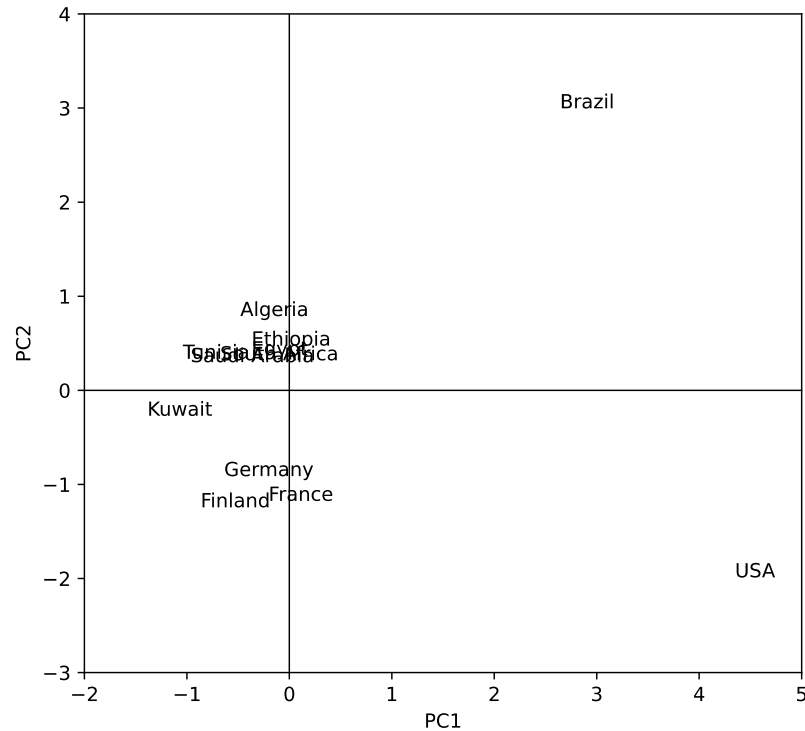
$$\ell_{ij} = \mathbf{W}_{ij} \sqrt{\frac{\text{Var}(Z_j)}{\text{Var}(X_i)}} = \mathbf{W}_{ij} \sqrt{\frac{\lambda_j}{\text{Var}(X_i)}}$$

which describes how much the variable **i** is **correlated** to the principal component **j**



Principal Component Analysis

We can use both plots to make **some qualitative observations** about the data:



- BR** Brazil is a large country (big area and big population) with a very high inflation (especially in 1991)
- US** USA is also a large country but with controlled inflation
- EU** The European countries are small and have a high GNB per capita
- US** USA has a very negative external trade, so is on the opposite direction to EXTTR

A practical example : Eigenfaces

Suppose we had a dataset with all photos from a class trombinoscope :

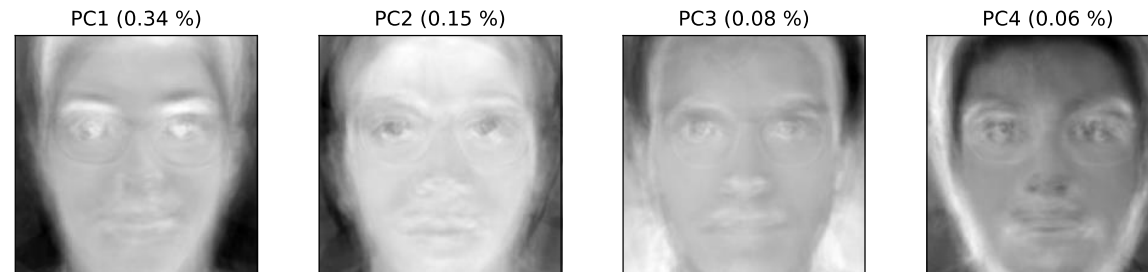
$$\mathcal{D} = \left\{ \begin{array}{c} \text{img 1} \quad \text{img 2} \quad \text{img 3} \quad \text{img 4} \quad \text{img 5} \quad \text{img 6} \quad \dots \end{array} \right\}$$

Flattening the matrices, we end up with $\mathbf{X} \in \mathbb{R}^{75 \times 16384}$

128x128 pixels

Can we use PCA to get lower-dimensional representations of the photos?

We can calculate the covariance matrix and get its eigenvectors, which are “eigenfaces”



A practical example : Eigenfaces

Then, we would like to check if we can write


$$\text{Target Face} \approx s_1 \times \text{Eigenface}_1 + s_2 \times \text{Eigenface}_2 + s_3 \times \text{Eigenface}_3 + s_4 \times \text{Eigenface}_4$$

↳ Note that if this turns out to be a good approximation, then it means that each photo can be encoded with just 4 numbers (i.e., the weights) instead of 16384

But what would be the **most appropriate values** for these weights? The projection!

$$\begin{bmatrix} s_1 & s_2 & s_3 & s_4 \end{bmatrix} = \mathbf{x}^T \begin{bmatrix} | & | & | & | \\ \mathbf{q}_1 & \mathbf{q}_2 & \mathbf{q}_3 & \mathbf{q}_4 \\ | & | & | & | \end{bmatrix}$$

We should then compare the reconstruction based on 4 eigenfaces vs original photo

A practical example : Eigenfaces

The result with only four eigenfaces is disappointing...

original photo



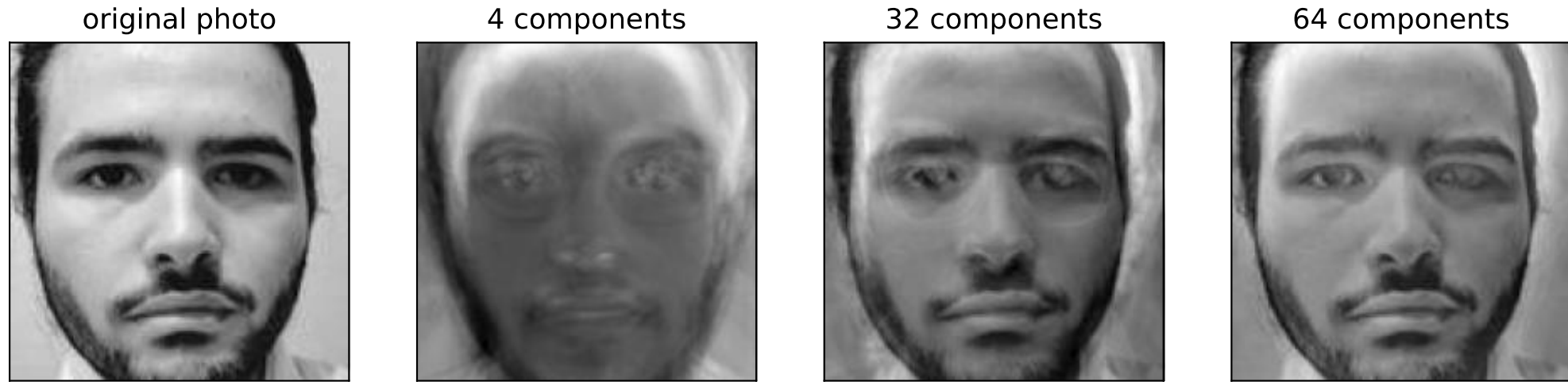
reconstruction with 4 components



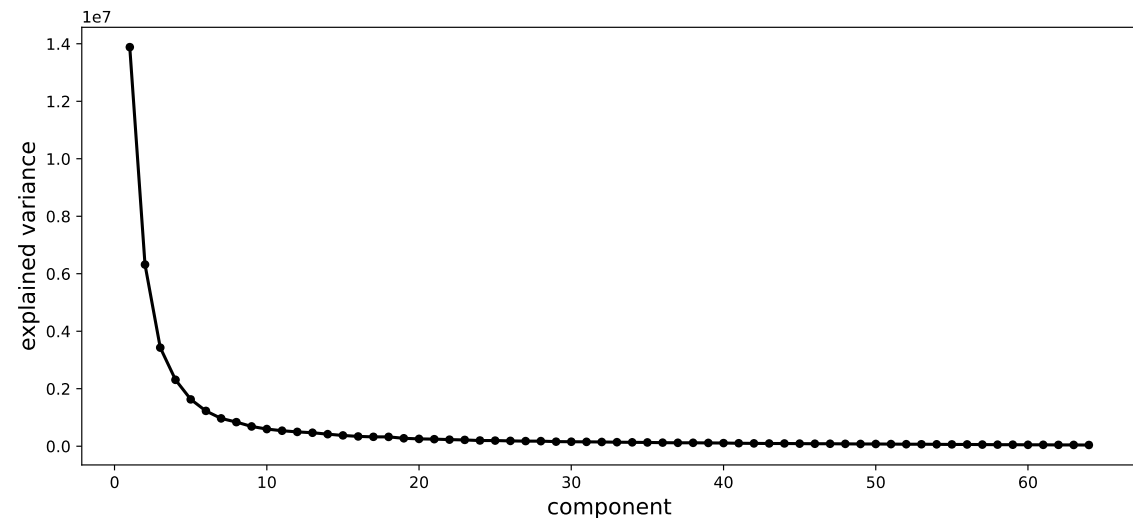
But what if we tested with more?

A practical example : Eigenfaces

The result with only four eigenfaces is disappointing... but what if had more?

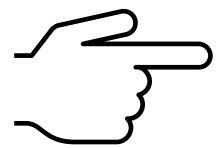


It is interesting to note that although the eigenvalues decay very quickly, we need to add many components in the decomposition to get a good visible result



- Some important mathematical tools

- Principal Component Analysis (PCA)

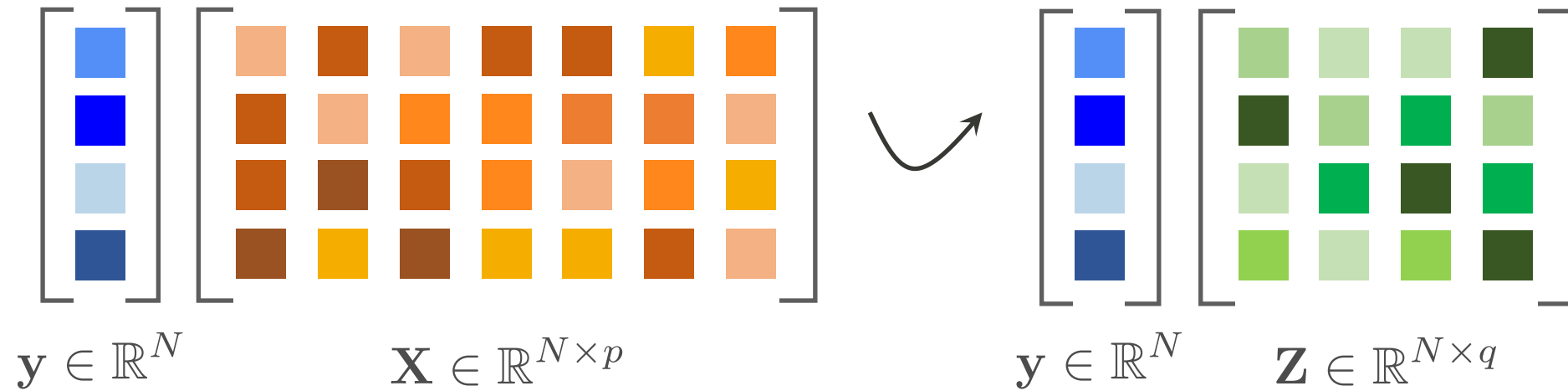


- Principal Component Regression (PCR)

- Some practical issues

Principal components regression

We will also suppose $\mathbf{y}$ has been re-centered so that the **intercept is zero** (why?)



We can do linear regression using a few principal components as features

$$\mathbf{y} = \mathbf{Z}\boldsymbol{\gamma} + \boldsymbol{\varepsilon} = (\mathbf{X}\mathbf{W}_q)\boldsymbol{\gamma} + \boldsymbol{\varepsilon} = \mathbf{X}(\mathbf{W}_q\boldsymbol{\gamma}) + \boldsymbol{\varepsilon}$$

and can relate the parameters of each model:

$$\boxed{\boldsymbol{\beta} = \mathbf{W}_q\boldsymbol{\gamma}} \quad \begin{array}{l} \boldsymbol{\beta} \in \mathbb{R}^p \\ \boldsymbol{\gamma} \in \mathbb{R}^q \end{array}$$

Principal components regression

Workflow for doing PCR on a dataset $\mathcal{D} = \left\{ (\mathbf{x}_i, y_i) \right\}_{i=1}^{i=N}$ with $\mathbf{x}_i \in \mathbb{R}^p$

- Choose the **dimension q** of the subspace to project the datapoints
- **Re-center** data points

$$\{\mathbf{x}_i\} \rightarrow \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i = \Delta \rightarrow \{\mathbf{x}'_i = \mathbf{x}_i - \Delta\} \rightarrow \frac{1}{N} \sum_{i=1}^N \mathbf{x}'_i = 0$$

Principal components regression

Workflow for doing PCR on a dataset $\mathcal{D} = \left\{ (\mathbf{x}_i, y_i) \right\}_{i=1}^{i=N}$ with $\mathbf{x}_i \in \mathbb{R}^p$

- Choose the **dimension q** of the subspace to project the datapoints
- **Re-center** data points

$$\{\mathbf{x}_i\} \rightarrow \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i = \Delta \rightarrow \{\mathbf{x}'_i = \mathbf{x}_i - \Delta\} \rightarrow \frac{1}{N} \sum_{i=1}^N \mathbf{x}'_i = 0$$



You're using information from the whole dataset!

Principal components regression

Workflow for doing PCR on a dataset $\mathcal{D} = \left\{ (\mathbf{x}_i, y_i) \right\}_{i=1}^{i=N}$ with $\mathbf{x}_i \in \mathbb{R}^p$

- Choose the **dimension q** of the subspace to project the datapoints
- Split the dataset into **training** and **test** partitions $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$

 Learn to **re-center** data points using only the training set

$$\{\mathbf{x}_i\} \rightarrow \frac{1}{N_{\text{train}}} \sum_{\mathcal{D}_{\text{train}}} \mathbf{x}_i = \Delta_{\text{train}} \rightarrow \{\mathbf{x}'_i = \mathbf{x}_i - \Delta_{\text{train}}\}$$

$$\frac{1}{N_{\text{train}}} \sum_{\mathcal{D}_{\text{train}}} \mathbf{x}'_i = 0$$

(By definition)

$$\frac{1}{N_{\text{test}}} \sum_{\mathcal{D}_{\text{test}}} \mathbf{x}'_i \approx 0$$

(By assumption)

Principal components regression

Workflow for doing PCR on a dataset $\mathcal{D} = \left\{ (\mathbf{x}_i, y_i) \right\}_{i=1}^{i=N}$ with $\mathbf{x}_i \in \mathbb{R}^p$

- Choose the **dimension q** of the subspace to project the datapoints
- Split the dataset into **training** and **test** partitions $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$

 Learn to **re-center** data points using only the training set

- Estimate the **covariance matrix** $\hat{\Sigma}_{\text{train}}$ of the re-centered samples from $\mathcal{D}_{\text{train}}$
- Get q-top **eigenvectors** of $\hat{\Sigma}_{\text{train}}$ and use to define the projector matrix $\mathbf{W}_{\text{train}}^{(q)}$
- **Project** data points from $\mathcal{D}_{\text{train}}$ to get the corresponding $\mathbf{z}_i = \left(\mathbf{W}_{\text{train}}^{(q)} \right)^\top \mathbf{x}'_i$

$$\mathcal{D}_{\text{train}} = \left\{ (\mathbf{x}'_i, y_i) \right\} \Rightarrow \mathcal{D}_{\text{train}}^{(q)} = \left\{ (\mathbf{z}_i, y_i) \right\}$$

- **Regress** on the observations to get $\hat{\gamma}$ and then the $\hat{\beta} = \mathbf{W}_{\text{train}}^{(q)} \hat{\gamma}$

Principal components regression

Workflow for doing PCR on a dataset $\mathcal{D} = \left\{ (\mathbf{x}_i, y_i) \right\}_{i=1}^{i=N}$ with $\mathbf{x}_i \in \mathbb{R}^p$



Evaluate the **MSE** on $\mathcal{D}_{\text{test}}$ being careful to use the pre-processing learned in training set

$$\mathbf{x}_i \in \mathcal{D}_{\text{test}} \rightarrow \mathbf{x}'_i = \mathbf{x}_i - \Delta_{\text{train}} \rightarrow \mathbf{z}_i = \left(\mathbf{W}_{\text{train}}^{(q)} \right)^\top \mathbf{x}'_i \rightarrow \mathbf{z}_i \in \mathcal{D}_{\text{test}}^{(q)}$$

both learned from training dataset

$$\text{MSE}(\hat{\gamma}) = \frac{1}{N_{\text{test}}} \sum_{\mathcal{D}_{\text{test}}^{(q)}} \left(y_i - \mathbf{z}_i^\top \hat{\gamma} \right)^2$$

Be sure to follow this procedure, otherwise you risk having exceedingly **optimistic** results

Principal components regression

Example of a PCR with **data leakage**

```
# reducing dimension before
pca = PCA(n_components=2)
X_red = pca.fit_transform(np.concatenate([X_train, X_test]))
X_red_train = X_red[:len(X_train)]
X_red_test = X_red[len(X_train):]
lr = LinearRegression()
lr.fit(X_red_train, y_train)
print(lr.score(X_red_test, y_test))
```

0.97

```
# reducing dimension at the right moment
pca = PCA(n_components=2)
X_red_train = pca.fit_transform(X_train)
X_red_test = pca.transform(X_test)
lr = LinearRegression()
lr.fit(X_red_train, y_train)
print(lr.score(X_red_test, y_test))
```

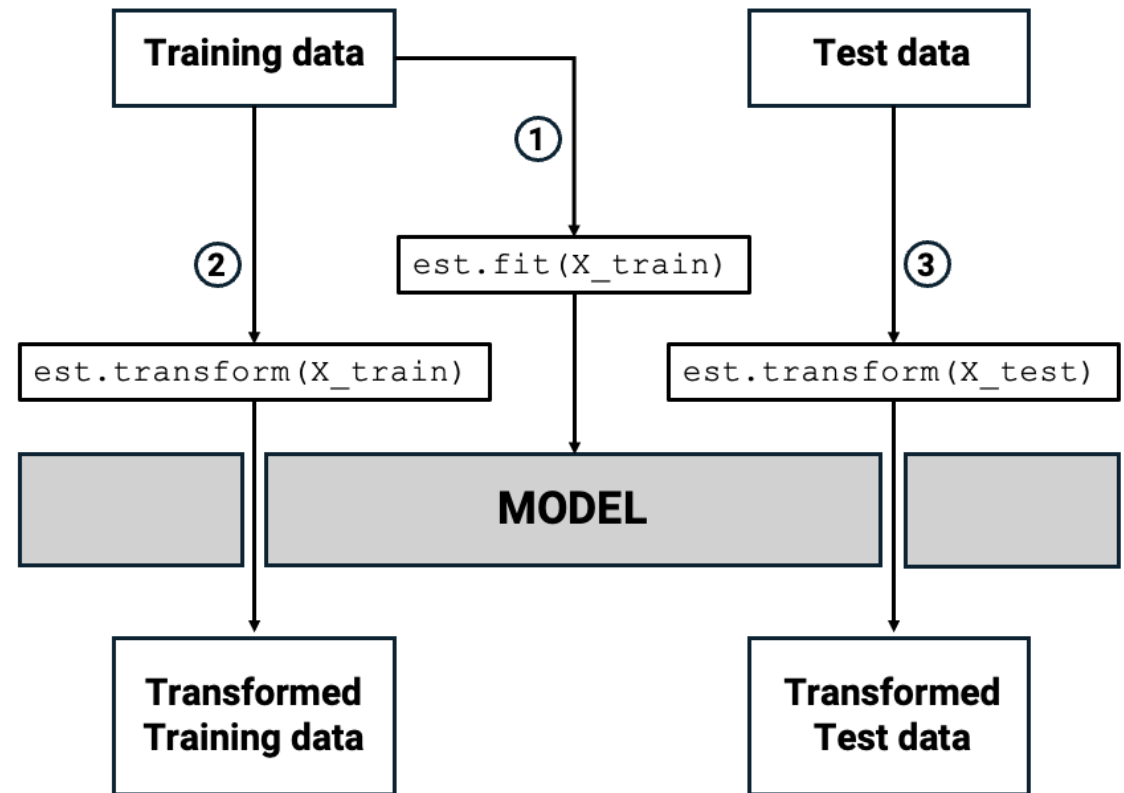
0.01

Principal components regression : in practice

```
scaler = StandardScaler()
pca = PCA(n_components=4)
lr = LinearRegression()

cv = KFold(n_splits=5)
scores = []
for train_idx, test_idx in cv.split(X, y):
    # split the train and test partitions
    X_train, y_train = X[train_idx], y[train_idx]
    X_test, y_test = X[test_idx], y[test_idx]
    # scale the data
    scaler.fit(X_train)
    X_train_scaled = scaler.transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    # reduce the dimensionality
    pca.fit(X_train_scaled)
    X_train_reduced = pca.transform(X_train_scaled)
    X_test_reduced = pca.transform(X_test_scaled)
    # fit linear regression and evaluate error
    lr.fit(X_train_reduced, y_train)
    scores.append(lr.score(X_test_reduced, y_test))
print(np.mean(scores))
```

The transformer API in scikit-learn



Principal components regression : in practice

```
scaler = StandardScaler()
pca = PCA(n_components=4)
lr = LinearRegression()

cv = KFold(n_splits=5)
scores = []
for train_idx, test_idx in cv.split(X, y):
    # split the train and test partitions
    X_train, y_train = X[train_idx], y[train_idx]
    X_test, y_test = X[test_idx], y[test_idx]
    # scale the data
    scaler.fit(X_train)
    X_train_scaled = scaler.transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    # reduce the dimensionality
    pca.fit(X_train_scaled)
    X_train_reduced = pca.transform(X_train_scaled)
    X_test_reduced = pca.transform(X_test_scaled)
    # fit linear regression and evaluate error
    lr.fit(X_train_reduced, y_train)
    scores.append(lr.score(X_test_reduced, y_test))
print(np.mean(scores))
```

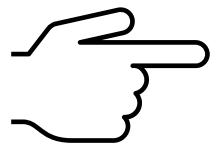
Pipelines make things
much less error prone!



```
est = make_pipeline(scaler, pca, lr)
```

```
cv = KFold(n_splits=5)
scores = []
for train_idx, test_idx in cv.split(X, y):
    # split the train and test partitions
    X_train, y_train = X[train_idx], y[train_idx]
    X_test, y_test = X[test_idx], y[test_idx]
    # fit the pipeline and evaluate error
    est.fit(X_train, y_train)
    scores.append(est.score(X_test, y_test))
print(np.mean(scores))
```

- Some important mathematical tools
- Principal Component Analysis (PCA)
- Principal Component Regression (PCR)



- Some practical issues

Some practical issues

What happens when $N \leq p$? i.e. when we have less samples than features

Some information we can use:

- The **rank of a matrix** is the dimension of the column space
- It is easy to show that $\text{rank}(\mathbf{X}) = \text{rank}(\mathbf{X}^\top \mathbf{X})$ -- use the null space to show this!
- We have that $\text{rank}(\mathbf{X}^\top \mathbf{X})$ is p minus the number of **zero eigenvalues**

Therefore, if $N \leq p$ then $\text{rank}(\mathbf{X}) = \text{rank}(\mathbf{X}^\top \mathbf{X}) \leq N$

And there are at least $p - N$ zero eigenvalues in the covariance matrix!

Some practical issues

What happens if the features have different scales? i.e. different variances

We know that the total variance of the data is split along the different eigenvalues

$$\text{tr}(\mathbf{\Sigma}) = \sum_{i=1}^p \text{Var}(X_i) = \sum_{j=1}^p \lambda_j$$

If feature F have much more variance than the others, then:

- The first component of PCA will be mostly aligned with the values of F, since it contains most of the total variance from the dataset
- Therefore, PC1 will have nearly zero loadings on all of its coordinates except the one for F

Some practical issues

What happens if the features have different scales? i.e. different variances

7.1.5. Wine recognition dataset

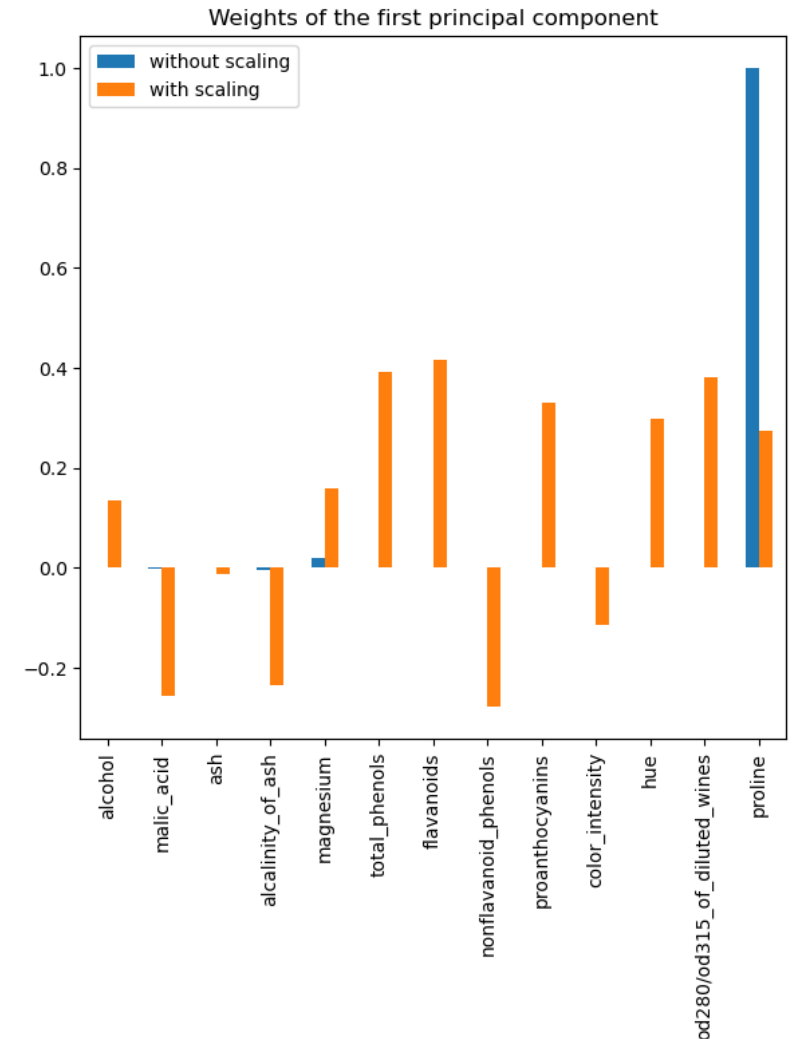
Data Set Characteristics:

Number of Instances:	178
Number of Attributes:	13 numeric, predictive attributes and the class
Attribute Information:	• Alcohol • Malic acid • Ash • Alcalinity of ash • Magnesium • Total phenols • Flavanoids • Nonflavanoid phenols • Proanthocyanins • Color intensity • Hue • OD280/OD315 of diluted wines • Proline

Summary Statistics:

Alcohol:	11.0	14.8	13.0	0.8
Malic Acid:	0.74	5.80	2.34	1.12
Ash:	1.36	3.23	2.36	0.27
Alcalinity of Ash:	10.6	30.0	19.5	3.3
Magnesium:	70.0	162.0	99.7	14.3
Total Phenols:	0.98	3.88	2.29	0.63
Flavanoids:	0.34	5.08	2.03	1.00
Nonflavanoid Phenols:	0.13	0.66	0.36	0.12
Proanthocyanins:	0.41	3.58	1.59	0.57
Colour Intensity:	1.3	13.0	5.1	2.3
Hue:	0.48	1.71	0.96	0.23
OD280/OD315 of diluted wines:	1.27	4.00	2.61	0.71
Proline:	278	1680	746	315

Note the difference in scale among the features for this dataset



Some practical issues

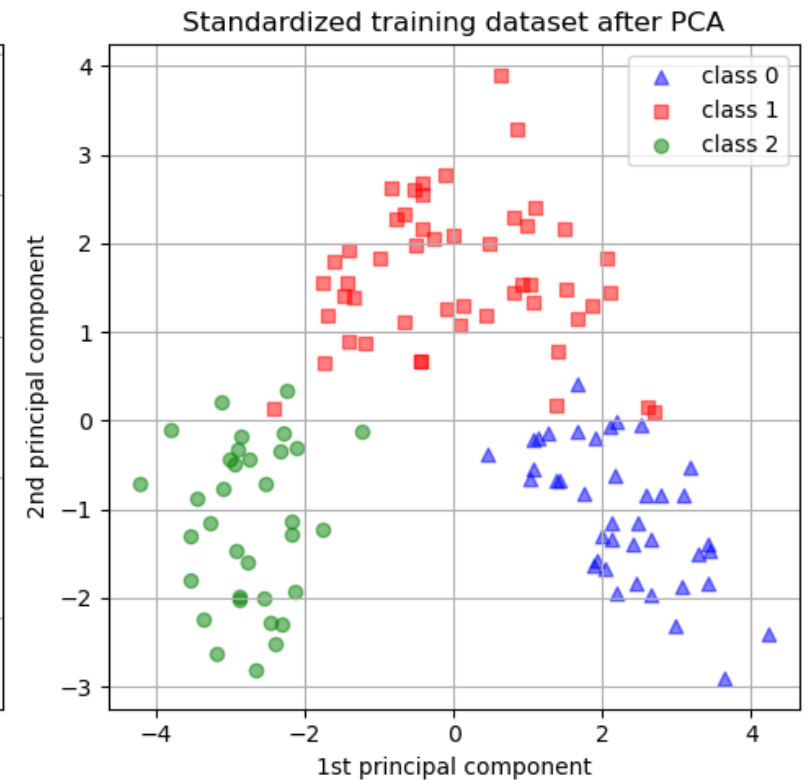
What happens if the features have different scales? i.e. different variances

Summary

Statistics:

Alcohol:	11.0	14.8	13.0	0.8
Malic Acid:	0.74	5.80	2.34	1.12
Ash:	1.36	3.23	2.36	0.27
Alcalinity of Ash:	10.6	30.0	19.5	3.3
Magnesium:	70.0	162.0	99.7	14.3
Total Phenols:	0.98	3.88	2.29	0.63
Flavanoids:	0.34	5.08	2.03	1.00
Nonflavanoid Phenols:	0.13	0.66	0.36	0.12
Proanthocyanins:	0.41	3.58	1.59	0.57
Colour Intensity:	1.3	13.0	5.1	2.3
Hue:	0.48	1.71	0.96	0.23
OD280/OD315 of diluted wines:	1.27	4.00	2.61	0.71
Proline:	278	1680	746	315

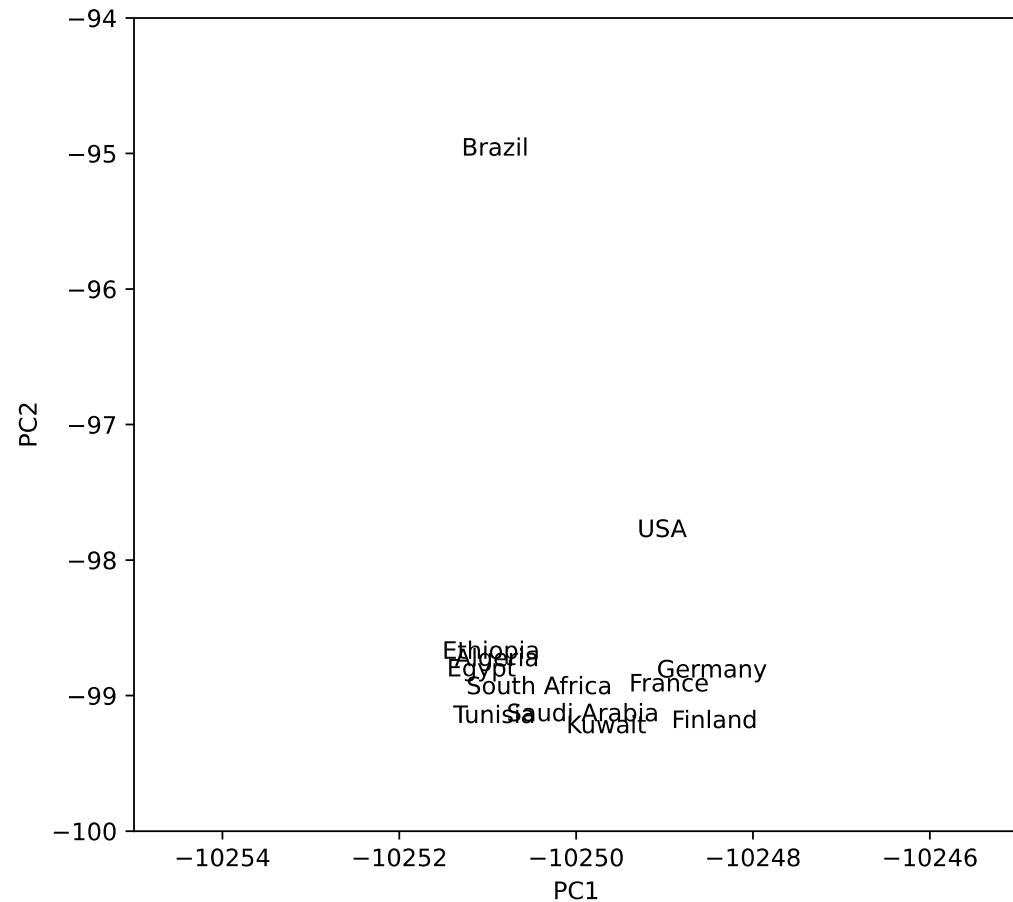
Note the difference in scale among the features for this dataset



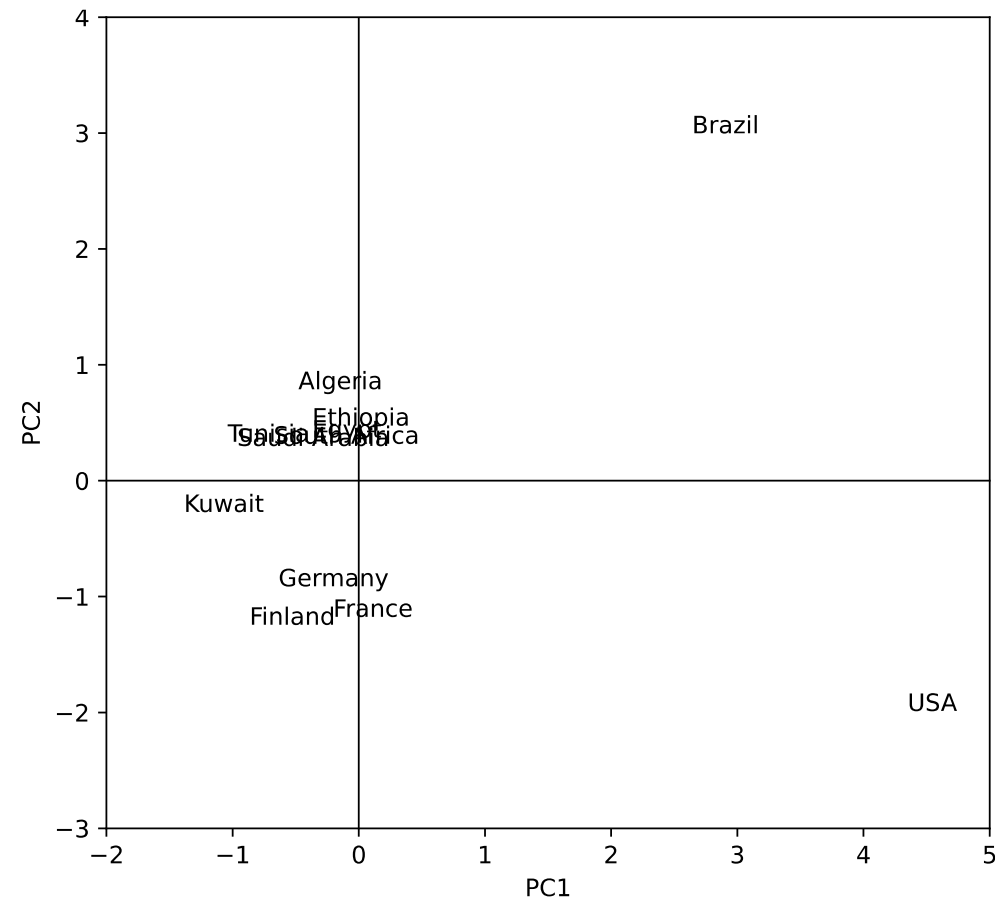
Some practical issues

What happens if the features have different scales? i.e. different variances

Without scaling



With scaling



Some practical issues

How do we calculate the PCA in practice?

Use **singular value decomposition!**

Every matrix can be decomposed as:

$$\underset{N \times p}{\mathbf{X}} = \underset{N \times p}{\mathbf{U}} \underset{p \times p}{\mathbf{D}} \underset{p \times p}{\mathbf{V}^\top}$$

- Where $\mathbf{D} = \text{diag}(d_1, \dots, d_p)$ with $d_1 \geq \dots \geq d_p \geq 0$
- Matrices $\mathbf{U}$ and $\mathbf{V}$ have orthonormal columns.

$$\mathbf{V} = \begin{bmatrix} | & & | \\ \mathbf{v}_1 & \dots & \mathbf{v}_p \\ | & & | \end{bmatrix}$$

principal components

$$\mathbf{U} = \begin{bmatrix} | & & | \\ \mathbf{u}_1 & \dots & \mathbf{u}_p \\ | & & | \end{bmatrix}$$

normalized principal scores

$$\frac{1}{N} d_i^2 = \text{the variance explained by } \mathbf{v}_i$$

(always remember to re-center the data matrix!)

Another practical example

Consider the `foodOfTheWorld` dataset

Three cooking websites were **scraped** to form a database with ~13k recipes

Each row contains the ingredients of a recipe

[illegible]

Another practical example

Consider the `foodOfTheWorld` dataset

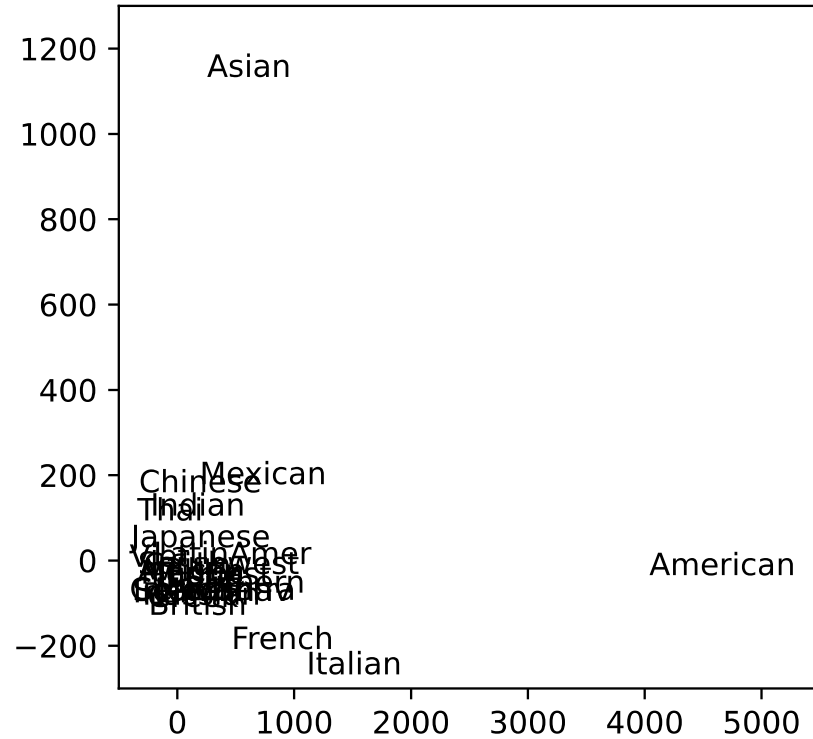
We get a contingency table with 26 rows (country) and 22 columns (ingredients)

position (i,j) indicates how many recipes of the country i had the ingredient j

	bell_pepper	butter	cayenne	cilantro	coriander	corn	cream	cumin	egg	fish	garlic	ginger	lime_juice	olive_oil	onion	rice	scallion	soy_sauce	tomato	turmeric	vegetable_oil	wheat
African	36	24	41	28	32	4	4	49	23	9	57	30	5	60	61	12	6	1	29	18	20	30
American	481	2219	653	289	139	516	1238	290	1738	201	1267	394	227	1512	1254	161	360	113	665	76	926	1574
Asian	155	69	345	244	157	59	42	98	161	228	565	576	195	78	167	482	453	588	68	94	426	112
Cajun	50	53	82	1	2	15	18	5	37	7	71	7	4	42	102	38	42	3	46	0	50	45
LatinAmer	40	56	125	75	12	79	44	58	51	8	137	2	41	75	131	15	34	3	100	2	50	47
Chinese	18	11	49	37	9	19	5	2	35	13	116	140	5	3	24	86	107	150	8	2	68	29
Russian	17	88	7	3	2	6	49	3	74	5	33	3	1	24	56	7	6	0	20	0	29	72
British	4	137	3	4	6	5	84	6	109	6	19	10	3	27	36	1	3	0	9	5	29	127
French	52	488	38	7	23	27	269	16	433	68	272	39	8	307	236	18	21	3	128	8	105	355
German	1	29	0	0	0	5	8	1	22	1	9	3	0	3	25	1	1	0	6	0	12	26
Greek	27	45	11	2	4	1	12	8	49	12	100	1	0	171	82	15	22	0	75	0	6	54
Indian	45	52	125	79	128	12	18	160	34	11	114	117	26	40	113	51	25	2	58	127	120	38
Irish	3	51	1	0	2	2	23	0	40	3	8	2	0	3	22	0	4	0	7	0	19	43
Italian	242	474	113	4	12	80	259	7	452	125	774	14	3	1124	480	92	47	5	518	2	90	430
Japanese	15	8	15	8	5	1	4	3	16	23	27	48	13	9	11	62	49	83	2	3	54	16
Jewish	18	98	21	18	15	4	28	20	190	14	60	20	3	80	97	3	22	0	30	7	87	156
MedTerra	52	41	30	13	15	7	15	30	56	39	146	5	1	230	112	20	20	0	101	8	22	60
Mexican	71	93	441	251	40	220	108	140	106	24	353	4	159	179	378	23	85	4	305	2	165	127
MidEast	41	53	46	40	40	8	11	85	38	6	116	15	6	149	88	19	31	4	58	22	30	94
Marroc	44	26	40	43	46	2	4	75	8	10	63	37	3	100	68	3	6	0	33	21	13	57
Scandinav	3	49	0	1	8	2	29	6	38	2	4	5	2	10	22	0	3	0	3	4	24	27
Southern	63	200	84	5	6	103	71	15	144	2	89	17	4	47	100	16	49	2	33	4	66	168
Southwest	33	15	88	56	7	42	19	36	17	0	67	2	29	43	66	3	23	4	43	0	23	17
Iberic	100	42	70	30	4	14	30	28	79	24	167	6	15	183	126	24	9	1	95	3	36	46
Thai	27	11	77	67	69	6	6	54	13	89	93	64	61	12	26	66	47	38	22	44	67	12
Viet	8	5	30	24	7	2	1	2	9	51	47	16	30	2	19	31	22	15	3	8	31	3

Another practical example

Consider the `foodOfTheWorld` dataset

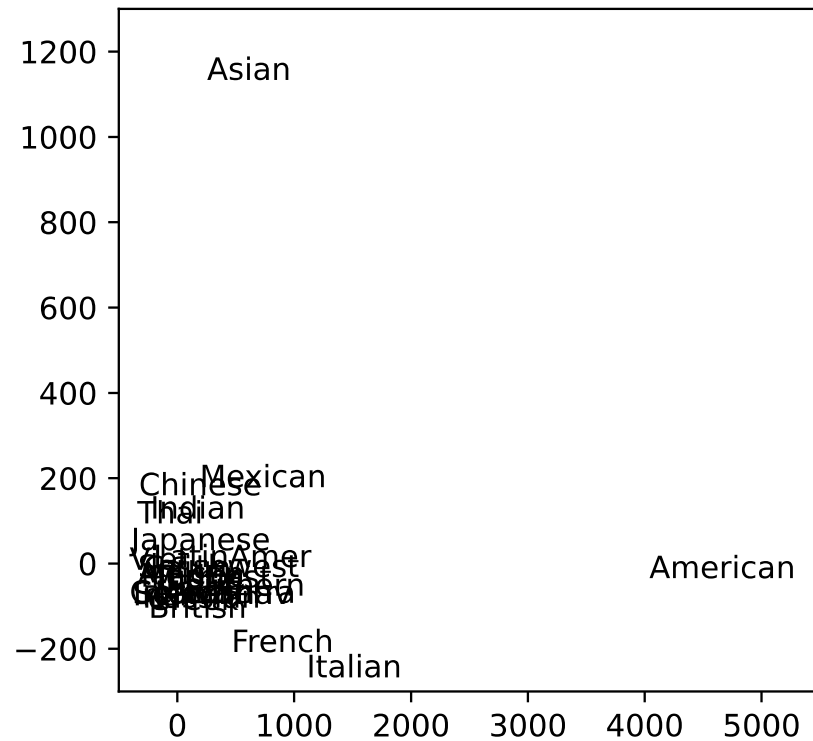


```
X = df.values
```

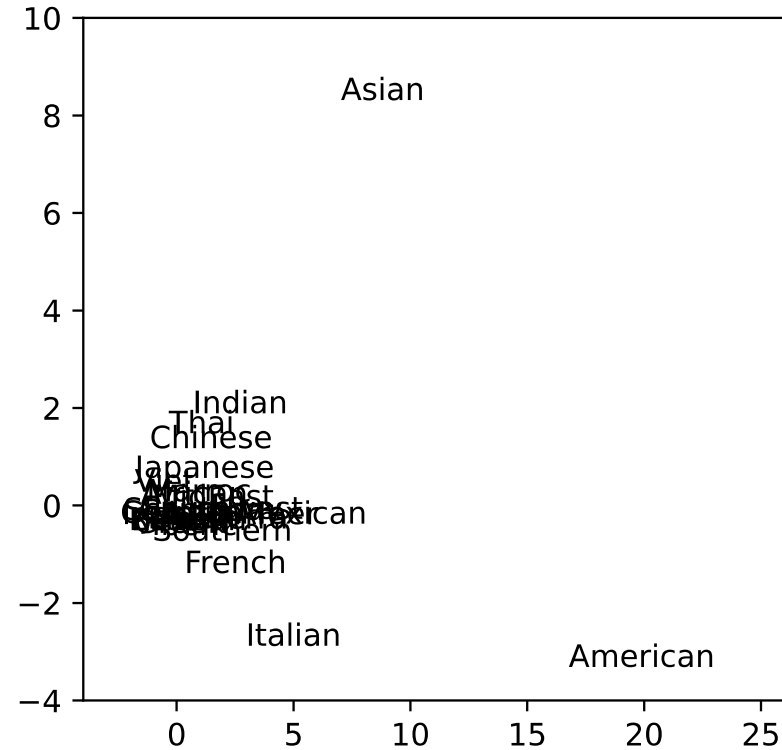
```
Z0 = pca.fit_transform(X)
```

Another practical example

Consider the `foodOfTheWorld` dataset



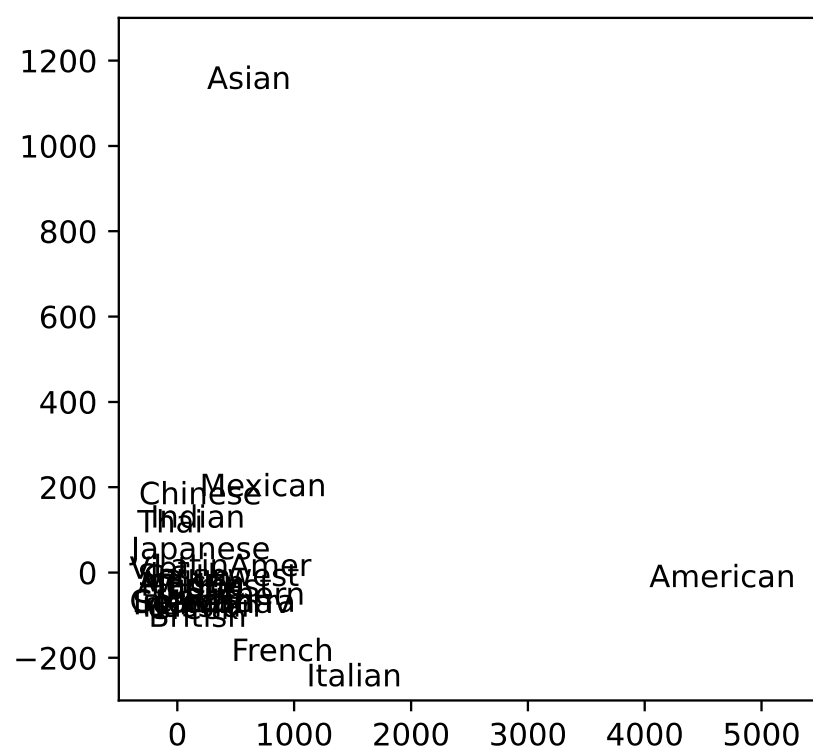
```
X = df.values  
Z0 = pca.fit_transform(X)
```



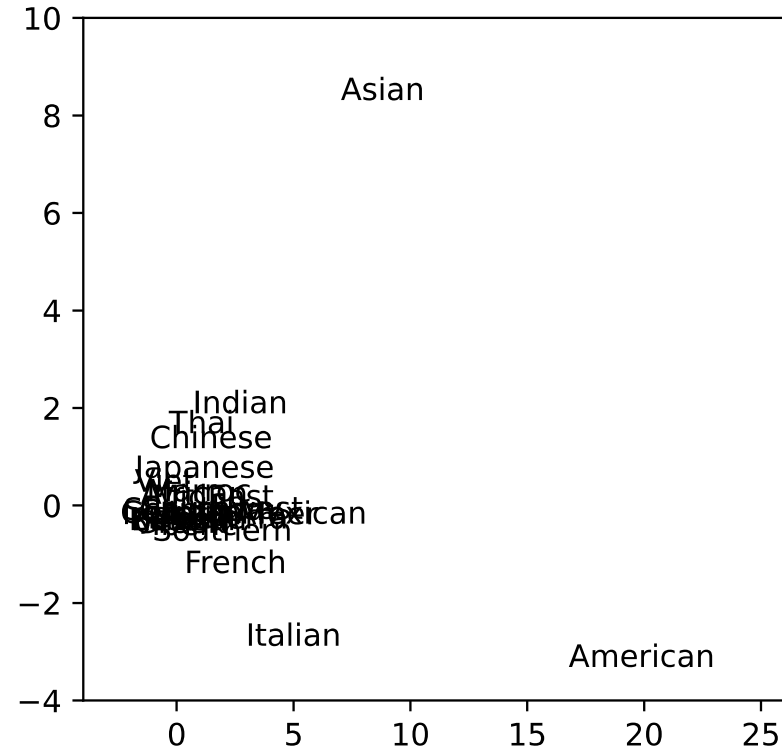
```
X = df.values  
est = make_pipeline(scaler, pca)  
Z1 = est.fit_transform(X)
```

Another practical example

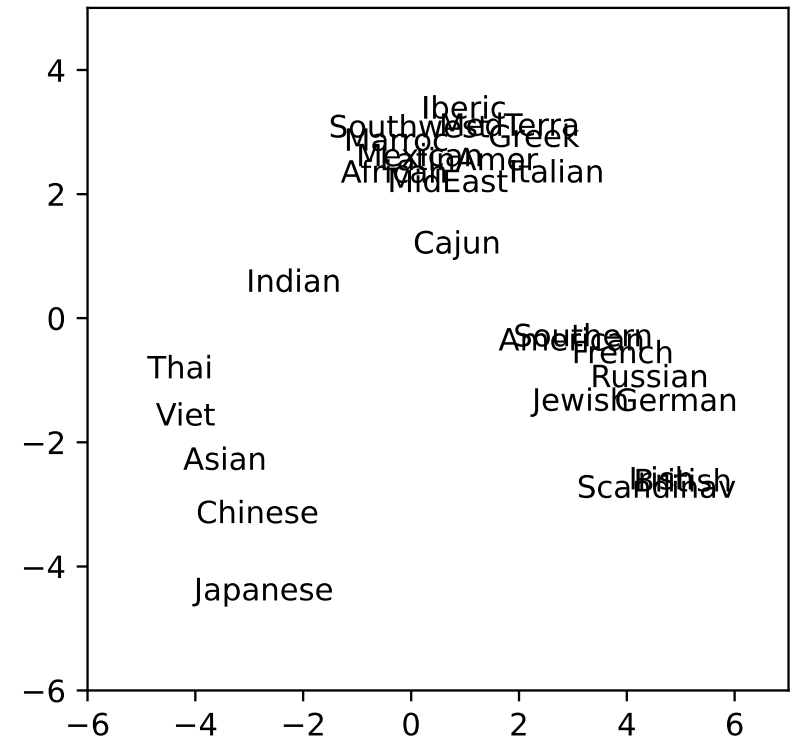
Consider the `foodOfTheWorld` dataset



```
X = df.values
Z0 = pca.fit_transform(X)
```

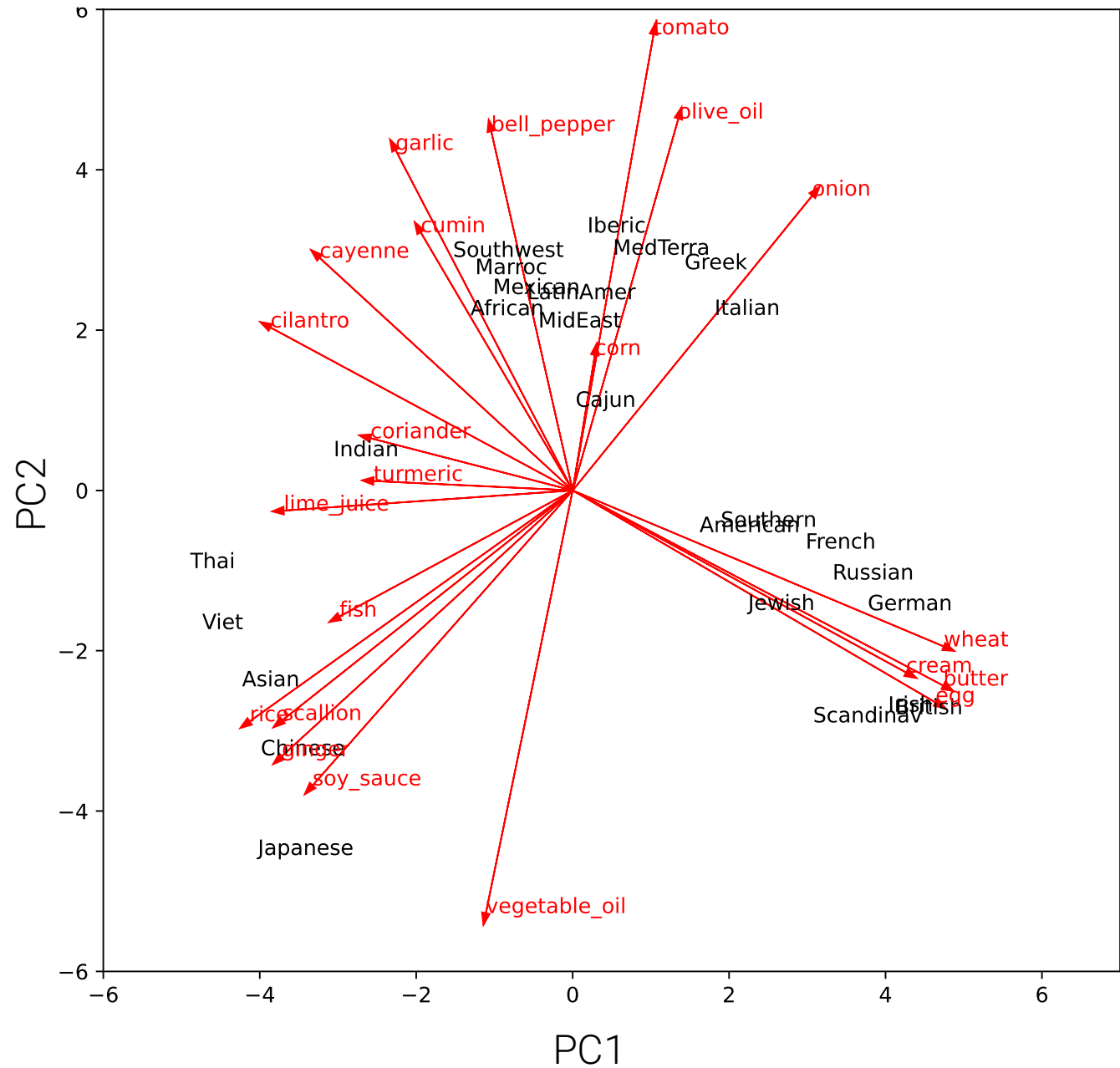


```
X = df.values
est = make_pipeline(scaler, pca)
Z1 = est.fit_transform(X)
```



```
X = df.values
rows = np.sum(X, axis=1)
X_ = X / rows[:, None]
est = make_pipeline(scaler, pca)
Z2 = est.fit_transform(X_)
```

Another practical example



Some practical issues

Consider the `foodOfTheWorld` dataset available here:

<https://github.com/HerveAbdi/data4PCCAR/blob/master/data/foodOfTheWorld.rda>

If you want to delve further in this dataset, see blog post using R to explore the dataset

<https://www.r-bloggers.com/2015/12/a-visualization-of-world-cuisines/>

Or even this article studying how similar flavors combine

[Ahn et al. “Flavor network and the principles of food pairing” \(2011\)](#)

